

dWallet Developer Guide

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

dWallet enables smart contracts to **control signing keys** on any blockchain. Your program determines what gets signed – the Ika network performs the distributed signing via 2PC-MPC.

How It Works

1. **Create a dWallet** – the Ika network runs DKG and produces a public key
2. **Your program controls it** – transfer the dWallet authority to your program’s CPI authority PDA
3. **Approve messages** – when conditions are met, your program CPI-calls `approve_message`
4. **Network signs** – the Ika validator network produces the signature via 2PC-MPC
5. **Signature stored on-chain** – anyone can read the MessageApproval account to get the signature

```
// Your program decides when to sign
fn cast_vote(ctx: &DWalletContext, proposal: &Proposal) -> ProgramResult {
    if proposal.yes_votes >= proposal.quorum {
        ctx.approve_message(
            message_approval, dwallet, payer, system_program,
            proposal.message_hash, user_pubkey, signature_scheme, bump,
        )?;
    }
    Ok(())
}
```

What You'll Learn

- **Getting Started:** Install dependencies, create your first dWallet-controlled program
- **Tutorial:** Build a voting app where quorum triggers signing
- **On-Chain Integration:** dWallet accounts, message approval, CPI framework, gas deposits
- **gRPC API:** SubmitTransaction, request/response types
- **Testing:** Mollusk, LiteSVM, and E2E testing
- **Reference:** Instructions, accounts, events

Installation

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Prerequisites

- **Rust** (edition 2024): `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`
- **Solana CLI** 3.x+: `sh -c "$(curl -sSfL https://release.anza.xyz/stable/install)"`

Add Dependencies

For Pinocchio Programs

```
[dependencies]
ika-dwallet-pinocchio = { git = "https://github.com/dwallet-labs/ika-pre-alpha"
}
pinocchio = "0.10"
pinocchio-system = "0.5"
```

For Anchor Programs

```
[dependencies]
ika-dwallet-anchor = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
anchor-lang = "1"
```

Requires [Anchor CLI 1.x](#) for build/deploy tooling. See the [Anchor framework guide](#) for usage details.

For Off-Chain Clients (gRPC)

[dependencies]

```
ika-grpc = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }  
ika-dwallet-types = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }  
tokio = { version = "1", features = ["rt-multi-thread", "macros"] }
```

For SDK Types (Account Readers, PDA Helpers)

[dependencies]

```
ika-sdk-types = { package = "ika-solana-sdk-types", git =  
"https://github.com/dwallet-labs/ika-pre-alpha" }
```

Pre-Alpha Environment

Resource	Endpoint
dWallet gRPC	https://pre-alpha-dev-1.ika.ika-network.net:443
Solana RPC	https://api.devnet.solana.com
Program ID	87W54kGYFQ1rgWqMeu4XTPHWXWmXSQCcj8vCTfiq1oY

No local validator or MPC node setup needed – just connect to devnet and start building.

Quick Start

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided "as is" without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Build your first dWallet-controlled program in 5 minutes.

1. Create a Solana Program

Pick your framework. All four produce interoperable programs:

Pinocchio (maximum CU efficiency, `no_std`):

```
[dependencies]
ika-dwallet-pinocchio = { git = "https://github.com/dwallet-labs/ika-pre-alpha"
}
pinocchio = "0.10"
pinocchio-system = "0.5"
```

Quasar (zero-copy + declarative validation, `no_std`):

```
[dependencies]
ika-dwallet-quasar = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
quasar-lang = { git = "https://github.com/blueshift-gg/quasar", branch =
"master" }
solana-address = { version = "2.4", features = ["curve25519"] }
```

Anchor v1 (easiest, declarative):

```
[dependencies]
ika-dwallet-anchor = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
anchor-lang = "1"
```

Native (standard `solana-program`, no framework):

```
[dependencies]
```

```
ika-dwallet-native = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
solana-program = "2.2"
```

All require:

```
[lib]
```

```
crate-type = ["cdylib", "lib"]
```

2. Set Up the CPI Context

```
#![no_std]
extern crate alloc;

use pinocchio::{entrypoint, AccountView, Address, ProgramResult};
use ika_dwallet_pinocchio::DWalletContext;

entrypoint!(process_instruction);
pinocchio::nostd_panic_handler!();

pub const ID: Address = Address::new_from_array([5u8; 32]);
```

The `DWalletContext` provides CPI methods for interacting with the dWallet program:

```
let ctx = DWalletContext {
    dwallet_program,
    cpi_authority,
    caller_program,
    cpi_authority_bump,
};
```

3. Approve a Message

When your program's conditions are met, call `approve_message` via CPI:

```
ctx.approve_message(
    message_approval, // writable PDA to create
    dwallet,          // the dWallet account
    payer,             // rent payer
    system_program,    // system program
    message_hash,      // 32-byte hash of the message to sign
    user_pubkey,       // 32-byte user public key
    signature_scheme,  // 0=Ed25519, 1=Secp256k1, 2=Secp256r1
    bump,              // MessageApproval PDA bump
)?;
```

This creates a `MessageApproval` PDA on-chain. The Ika network detects it and produces a signature.

4. Transfer dWallet Authority

Before your program can approve messages, the dWallet's authority must point to your program's CPI authority PDA:

```
// Derive the CPI authority PDA
// Seeds: [b"__ika_cpi_authority"], program_id = YOUR_PROGRAM_ID
let (cpi_authority, _bump) = Address::find_program_address(
    &[b"__ika_cpi_authority"],
    &your_program_id,
);

// Transfer ownership (called by current authority, typically the dWallet creator)
ctx.transfer_dwallet(dwallet, cpi_authority.as_array())?;
```

5. Read the Signature

After the network signs, the `MessageApproval` account contains the signature:

Offset	Field	Size
139	status	1
140	signature_len	2
142	signature	up to 128

Status values:

- 0 = Pending (awaiting signature)
- 1 = Signed (signature available)

What Happens Under the Hood

1. Your program calls `approve_message` via CPI -> creates a `MessageApproval` PDA (status = Pending)
2. The Ika network detects the `MessageApproval` account
3. The NOA (Network Operated Authority) signs the message using 2PC-MPC
4. The NOA calls `CommitSignature` to write the signature on-chain (status = Signed)

5. Anyone can read the signature from the `MessageApproval` account

In pre-alpha mode, step 3 uses a mock signer. All 11 protocol operations are supported (DKG, Sign, Presign, PresignForDWallet, ImportedKeyVerification, ReEncryptShare, MakeSharePublic, FutureSign, SignWithPartialUserSig, and more) across all 4 curves and 7 signature schemes.

Pre-Alpha Environment

Resource	Endpoint
dWallet gRPC	<code>https://pre-alpha-dev-1.ika.ika-network.net:443</code>
Solana Network	Devnet (<code>https://api.devnet.solana.com</code>)
Program ID	<code>87W54kGYFQ1rgWqMeu4XTPHWXWmXSQCcj8vCTfiq1oY</code>

Core Concepts

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

dWallet

A **dWallet** is a distributed signing key controlled by a Solana account. The on-chain `DWallet` account stores the public key, curve type, and authority. The private key never exists in one place – it is split between the user and the Ika validator network via 2PC-MPC (two-party computation with multi-party computation).

DWallet account (on Solana):

```

  authority(32)      -- who can approve signing
  curve(2)           -- u16 LE: Secp256k1(0), Secp256r1(1), Curve25519(2),
Ristretto(3)
  state(1)           -- DKGInProgress(0), Active(1), Frozen(2)
  public_key_len(1)  -- actual public key length (32 or 33)
  public_key(65)     -- the dWallet's public key (padded to 65 bytes)
  created_epoch(8)   -- epoch when created
  noa_public_key(32) -- NOA Ed25519 key used during DKG
  is_imported(1)     -- whether the key was imported (vs created via DKG)
  bump(1)            -- PDA bump seed
  _reserved(8)       -- reserved for future use

```

Attestation data (DKG output, proofs, etc.) is stored in separate `DWalletAttestation` PDAs, not inline in the `DWallet` account.

A dWallet can sign transactions on **any blockchain** – Bitcoin, Ethereum, Solana, etc. The curve and signature scheme determine which chains are compatible.

Authority

The **authority** of a dWallet controls who can approve messages for signing. It can be:

- A **user wallet** (direct signer) – the user calls `approve_message` directly
- A **CPI authority PDA** – a program controls the dWallet and approves messages via CPI

Transferring authority is done via the `TransferOwnership` instruction.

CPI Authority PDA

Every program that wants to control a dWallet derives a **CPI authority PDA**:

```
Seeds: [b"__ika_cpi_authority"]
Program: YOUR_PROGRAM_ID
```

When a dWallet's authority is set to your program's CPI authority PDA, only your program can approve messages for that dWallet. The dWallet program verifies the CPI call chain to ensure the correct program is calling.

Message Approval

A **MessageApproval** is a PDA that represents a request to sign a specific message. When your program calls `approve_message`, it creates this PDA:

MessageApproval PDA:

```
Seeds: ["dwallet", chunks..., "message_approval", &scheme_u16_le,
&message_digest, [&meta_digest]]
Program: DWALLET_PROGRAM_ID
```

Fields:

<code>dwallet(32)</code>	-- the dWallet to sign with
<code>message_digest(32)</code>	-- keccak256 digest of the message
<code>message_metadata_digest(32)</code>	-- keccak256 digest of metadata (zero if none)
<code>approver(32)</code>	-- dWallet authority who authorized signing
<code>user_pubkey(32)</code>	-- user's public key
<code>signature_scheme(2)</code>	-- DWalletSignatureScheme (u16 LE, values 0-6)
<code>epoch(8)</code>	-- epoch when approved
<code>status(1)</code>	-- Pending(0) or Signed(1)
<code>signature_len(2)</code>	-- length of signature bytes
<code>signature(128)</code>	-- the produced signature (padded)
<code>bump(1)</code>	-- PDA bump
<code>_reserved(8)</code>	-- reserved

The Ika network monitors for new `MessageApproval` accounts and produces signatures for those with status = Pending.

NOA (Network Operated Authority)

The **NOA** is a special keypair operated by the Ika network. In the pre-alpha, this is a single mock signer. In production, the NOA's actions are backed by MPC consensus across all validators.

The NOA:

- Initializes the dWallet program state (`DWalletCoordinator`, `NetworkEncryptionKey`)
- Commits new dWallets after DKG (`CommitDWallet`)
- Commits signatures after signing (`CommitSignature`)
- Commits attestation PDAs (`CommitFutureSign` , `CommitEncryptedUserSecretKeyShare` , `CommitPublicUserSecretKeyShare`)
- Handles network DKG (`CommitNetworkDKG`) and key reconfiguration (`CommitNetworkKeyReconfiguration`)

Presign

A **presign** is a precomputed partial signature that speeds up the signing process. Presigns are generated in advance and consumed during signing.

There are two types:

- **Global presigns** – can be used with any non-imported dWallet (allocated via `Presign` request, uses `signature_algorithm`)
- **dWallet-specific presigns** – bound to a specific dWallet by `dwallet_public_key` (allocated via `PresignForDWallet` request, required for imported ECDSA keys)

Presigns are managed via the gRPC API and returned as `Attestation(NetworkSignedAttestation)` containing a `VersionedPresignDataAttestation` .

Gas Deposit

Programs that use dWallet instructions need a `GasDeposit` PDA. The deposit holds:

- **IKA balance**: For dWallet operation fees (DKG, signing, etc.)

- **SOL balance:** For NOA write-back transaction costs

Instructions: `CreateDeposit` (36), `TopUp` (37), `SettleGas` (38), `RequestWithdraw` (44), `Withdraw` (45).

Supported Curves and Signature Schemes

Curve	ID (u16)	Description	Mock DKG
Secp256k1	0	Bitcoin, Ethereum	Yes
Secp256r1	1	WebAuthn, secure enclaves	Yes
Curve25519	2	Solana, Sui, general Ed25519	Yes
Ristretto	3	Substrate, Polkadot	Yes

DWalletSignatureScheme (u16)

Combined (algorithm, hash) pair used for signing and message approval:

Variant	Index	Curve	Use For
EcdsaKeccak256	0	Secp256k1	Ethereum
EcdsaSha256	1	Secp256k1 / Secp256r1	Bitcoin (legacy) / WebAuthn
EcdsaDoubleSha256	2	Secp256k1	Bitcoin BIP143
TaprootSha256	3	Secp256k1	Bitcoin Taproot (BIP340)
EcdsaBlake2b256	4	Secp256k1	Zcash
EddsaSha512	5	Curve25519	Ed25519 (Solana, Sui)
SchnorrkelMerlin	6	Ristretto	Substrate, Polkadot (sr25519)

DWalletSignatureAlgorithm

Used by presign requests (presigns are per-algorithm, not per-scheme):

Variant	Value	Description
ECDSAsecp256k1	0	ECDSA on Secp256k1
ECDSAsecp256r1	1	ECDSA on Secp256r1
Taproot	2	Schnorr on Secp256k1

Variant	Value	Description
EdDSA	3	Ed25519 on Curve25519
Schnorrkel	4	sr25519 on Ristretto

DKG (Distributed Key Generation)

DKG is the process of creating a new dWallet. The user and the Ika network jointly generate a key pair such that:

- The user holds one share of the private key
- The network collectively holds the other share
- Neither party alone can produce a signature

The on-chain flow:

1. User submits DKG request via gRPC
2. Network runs 2PC-MPC DKG protocol
3. NOA calls `CommitDWallet` to create the on-chain dWallet account and its attestation PDA
4. The dWallet's authority is set to the requesting user

Tutorial: Voting dWallet

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

This tutorial builds a complete **voting-controlled dWallet** program on Solana. A proposal specifies a message to sign; when enough “yes” votes reach quorum, the program automatically approves the message for signing via CPI.

What You Will Build

A Solana program with two instructions:

Instruction	Discriminator	Description
<code>create_proposal</code>	0	Creates a proposal PDA with a target dWallet, message hash, and quorum
<code>cast_vote</code>	1	Records a vote; when quorum is reached, CPI-calls <code>approve_message</code>

How It Works

1. A dWallet is created and its authority transferred to the voting program’s CPI authority PDA
2. The creator submits a proposal referencing the dWallet, message hash, and required quorum
3. Voters cast yes/no votes – each vote creates a `VoteRecord` PDA (prevents double voting)
4. When yes votes reach quorum, the program automatically CPI-calls `approve_message` on the dWallet program

5. The Ika network detects the `MessageApproval` and produces a signature
6. The proposal status changes to `Approved`

Key Concepts Covered

- **CPI authority pattern** – transferring dWallet control to a program
- **dWalletContext** – the CPI wrapper for calling dWallet instructions
- **approve_message** – creating a MessageApproval PDA to trigger signing
- **PDA-based vote records** – preventing double voting via account existence
- **Mollusk tests** – unit testing individual instructions
- **E2E tests** – full lifecycle against Solana devnet and the pre-alpha gRPC service

Source Code

The complete example is at `chains/solana/examples/voting/`.

```
voting/  
  src/lib.rs           -- program logic (2 instructions)  
  tests/mollusk.rs     -- Mollusk instruction-level tests  
  e2e/src/main.rs      -- full E2E demo  
  Cargo.toml
```

Prerequisites

- [Installation](#) complete
- Familiarity with [Core Concepts](#)
- Basic Solana program development experience

Create the Program

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided "as is" without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Cargo.toml

```
[package]
name = "ika-example-voting"
version = "0.1.0"
edition = "2024"

[dependencies]
ika-dwallet-pinocchio = { git = "https://github.com/dwallet-labs/ika-pre-alpha"
}
pinocchio = "0.10"
pinocchio-system = "0.5"

[dev-dependencies]
mollusk-svm = "0.2"
solana-account = "2"
solana-instruction = "2"
solana-pubkey = "2"

[lib]
crate-type = ["cdylib", "lib"]
```

Key crates:

- **ika-dwallet-pinocchio** - DWalletContext CPI wrapper and CPI_AUTHORITY_SEED
- **pinocchio** - zero-copy Solana program framework
- **pinocchio-system** - CreateAccount CPI helper

lib.rs Skeleton

```
#![no_std]
extern crate alloc;

use pinocchio::{
    cpi::Signer,
    entrypoint,
    error::ProgramError,
    AccountView, Address, ProgramResult,
};
use pinocchio_system::instructions::CreateAccount;
use ika_dwallet_pinocchio::DWalletContext;

entrypoint!(process_instruction);
pinocchio::nostd_panic_handler!();

pub const ID: Address = Address::new_from_array([5u8; 32]);
```

Account Discriminators

```
const PROPOSAL_DISCRIMINATOR: u8 = 1;
const VOTE_RECORD_DISCRIMINATOR: u8 = 2;

const STATUS_OPEN: u8 = 0;
const STATUS_APPROVED: u8 = 1;
```

Proposal Account Layout

The Proposal PDA stores the dWallet reference, message hash, vote counts, and quorum:

Proposal PDA (seeds: ["proposal", proposal_id]):
195 bytes total (2-byte header + 193 bytes data)

Offset	Field	Size	Description
0	discriminator	1	1
1	version	1	1
2	proposal_id	32	Unique proposal identifier
34	dwallet	32	dWallet account pubkey
66	message_hash	32	Hash of the message to sign
98	user_pubkey	32	User public key for signing

Offset	Field	Size	Description
130	signature_scheme	1	Ed25519(0), Secp256k1(1), Secp256r1(2)
131	creator	32	Proposal creator pubkey
163	yes_votes	4	Yes vote count (LE u32)
167	no_votes	4	No vote count (LE u32)
171	quorum	4	Required yes votes (LE u32)
175	status	1	Open(0) or Approved(1)
176	msg_approval_bump	1	MessageApproval PDA bump
177	bump	1	Proposal PDA bump
178	_reserved	16	Reserved for future use

VoteRecord Account Layout

The VoteRecord PDA prevents double voting. Its existence proves the voter has already voted.

VoteRecord PDA (seeds: ["vote", proposal_id, voter]):
69 bytes total (2-byte header + 67 bytes data)

Offset	Field	Size	Description
0	discriminator	1	2
1	version	1	1
2	voter	32	Voter pubkey
34	proposal_id	32	Associated proposal
66	vote	1	Yes(1) or No(0)
67	bump	1	VoteRecord PDA bump

Instruction Dispatch

```
pub fn process_instruction(  
    program_id: &Address,  
    accounts: &[AccountView],  
    data: &[u8],  
) -> ProgramResult {  
    let (discriminator, rest) = data  
        .split_first()  
        .ok_or(ProgramError::InvalidInstructionData)?;  
  
    match *discriminator {  
        0 => create_proposal(program_id, accounts, rest),  
        1 => cast_vote(program_id, accounts, rest),  
        _ => Err(ProgramError::InvalidInstructionData),  
    }  
}
```

Rent Calculation

The dWallet program uses a simple rent formula:

```
fn minimum_balance(data_len: usize) -> u64 {  
    (data_len as u64 + 128) * 6960  
}
```

Next Step

With the program skeleton in place, the next chapter implements the `create_proposal` instruction and the [message approval flow](#).

Approve Messages

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

create_proposal Instruction

The `create_proposal` instruction creates a Proposal PDA that references a dWallet and the message to sign.

Instruction Data

Offset	Field	Size
0	proposal_id	32
32	message_hash	32
64	user_pubkey	32
96	signature_scheme	1
97	quorum	4
101	message_approval_bump	1
102	bump	1

Total: 103 bytes.

Accounts

#	Account	W	S	Description
0	proposal	yes	no	Proposal PDA (["proposal", proposal_id])

#	Account	W	S	Description
1	dwallet	no	no	dWallet account
2	creator	no	yes	Proposal creator (signer)
3	payer	yes	yes	Rent payer
4	system_program	no	no	System program

Implementation

```

fn create_proposal(
    program_id: &Address,
    accounts: &[AccountView],
    data: &[u8],
) -> ProgramResult {
    if data.len() < 103 {
        return Err(ProgramError::InvalidInstructionData);
    }
    let [proposal_account, _dwallet, creator, payer, _system_program, ..] =
accounts else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };

    if !creator.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    // Parse instruction data
    let proposal_id: [u8; 32] = data[0..32].try_into().unwrap();
    let message_hash: [u8; 32] = data[32..64].try_into().unwrap();
    let user_pubkey: [u8; 32] = data[64..96].try_into().unwrap();
    let signature_scheme = data[96];
    let quorum = u32::from_le_bytes(data[97..101].try_into().unwrap());
    let message_approval_bump = data[101];
    let bump = data[102];

    // Quorum must be at least 1
    if quorum == 0 {
        return Err(ProgramError::InvalidInstructionData);
    }

    // Create PDA with seeds ["proposal", proposal_id, bump]
    let bump_byte = [bump];
    let signer_seeds = [
        pinocchio::cpi::Seed::from(b"proposal" as &[u8]),
        pinocchio::cpi::Seed::from(proposal_id.as_ref()),
        pinocchio::cpi::Seed::from(bump_byte.as_ref()),
    ];
    let signer = Signer::from(&signer_seeds);

    CreateAccount {
        from: payer,
        to: proposal_account,
        lamports: minimum_balance(PROPOSAL_LEN),
        space: PROPOSAL_LEN as u64,
        owner: program_id,
    }
    .invoke_signed(&[signer])?;

    // Write all proposal fields into the account data
    let prop_data = unsafe { proposal_account.borrow_unchecked_mut() };
    prop_data[0] = PROPOSAL_DISCRIMINATOR;
    prop_data[1] = 1; // version
    // ... copy proposal_id, dwallet, message_hash, etc. ...

```

```
    Ok(() )  
}
```

The key points:

- The proposal stores the `message_hash` and `message_approval_bump` so the CPI call can construct the correct `MessageApproval` PDA later
- The `user_pubkey` and `signature_scheme` are passed through to `approve_message` when quorum is reached
- Quorum of zero is rejected

MessageApproval PDA

When quorum is reached, the program creates a `MessageApproval` PDA via CPI. This PDA is derived by the **dWallet program**, not the voting program:

```
Seeds: ["message_approval", dwallet_pubkey, message_hash]  
Program: DWALLET_PROGRAM_ID
```

Important: The `message_hash` you pass to `approve_message` is the **uniqueness key for the MessageApproval PDA** and must be computed as `keccak256(preimage)` regardless of which destination chain the dWallet will eventually sign for. The `dwallet` program treats it as opaque 32 bytes; using the same hash function (`keccak256` , with Solana's cheap on-chain `syscall`) for every chain keeps the `dwallet` program chain-agnostic.

The **digest the dwallet network actually signs** is a separate concern, controlled by the `hash_scheme` field on the gRPC `sign` request. For `Secp256k1` the network applies `hash_scheme(message)` and signs the resulting 32-byte digest via `sign_prehash` , so the produced signature is valid on whichever chain expects that exact hash function (`Keccak256` for EVM, `DoubleSHA256` for Bitcoin BIP143, etc.). For EVM the on-chain lookup hash and the signing digest happen to coincide; for Bitcoin they differ. The mock supports `hash_scheme` (it used to ignore it and always hash with SHA-256, which produced signatures that wouldn't verify on real EVM/Bitcoin nodes).

The dWallet program verifies:

1. The caller is a valid program (executable account)
2. The CPI authority PDA matches the dWallet's current authority
3. The CPI authority is signed (via `invoke_signed`)

Next Step

With proposals created, the next chapter implements [vote casting](#) and the quorum-triggered CPI.

Cast Votes

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

cast_vote Instruction

The `cast_vote` instruction records a vote and – when quorum is reached – triggers the CPI call to `approve_message`.

Instruction Data

Offset	Field	Size
0	proposal_id	32
32	vote	1
33	vote_record_bump	1
34	cpi_authority_bump	1

Total: 35 bytes. The `vote` field is `1` for yes and `0` for no.

Accounts (No Quorum Path)

#	Account	W	S	Description
0	proposal	yes	no	Proposal PDA
1	vote_record	yes	no	VoteRecord PDA (["vote", proposal_id, voter])
2	voter	no	yes	Voter (signer)
3	payer	yes	yes	Rent payer

#	Account	W	S	Description
4	system_program	no	no	System program

Additional Accounts (When Quorum Reached)

When the vote triggers quorum, 5 additional accounts are required:

#	Account	W	S	Description
5	message_approval	yes	no	MessageApproval PDA (to create via CPI)
6	dwallet	no	no	dWallet account
7	caller_program	no	no	This voting program (executable)
8	cpi_authority	no	no	CPI authority PDA (signer via invoke_signed)
9	dwallet_program	no	no	dWallet program

Implementation

The core logic:

```

fn cast_vote(
    program_id: &Address,
    accounts: &[AccountView],
    data: &[u8],
) -> ProgramResult {
    let proposal_id: [u8; 32] = data[0..32].try_into().unwrap();
    let vote = data[32];
    let vote_record_bump = data[33];
    let cpi_authority_bump = data[34];

    // 1. Verify proposal is open
    // 2. Create VoteRecord PDA (fails if already exists = double vote
    prevention)
    // 3. Update yes_votes or no_votes on the proposal
    // 4. If yes_votes >= quorum, trigger CPI

    // ... vote counting ...

    if yes_votes >= quorum {
        // Need additional accounts for CPI
        let message_approval = &accounts[5];
        let dwallet = &accounts[6];
        let caller_program = &accounts[7];
        let cpi_authority = &accounts[8];
        let dwallet_program = &accounts[9];

        // Build DWalletContext and call approve_message
        let ctx = DWalletContext {
            dwallet_program,
            cpi_authority,
            caller_program,
            cpi_authority_bump,
        };

        ctx.approve_message(
            message_approval,
            dwallet,
            payer,
            system_program,
            message_hash,
            user_pubkey,
            signature_scheme,
            message_approval_bump,
        )?;

        // Mark proposal as approved
        prop_data[PROP_STATUS] = STATUS_APPROVED;
    }

    Ok(())
}

```

Double Vote Prevention

The VoteRecord PDA uses seeds `["vote", proposal_id, voter]`. Since `CreateAccount` will fail if the account already exists (non-zero lamports), a voter cannot vote twice on the same proposal.

The CPI Call Chain

When quorum triggers `approve_message`, the call chain is:

```

Voting Program
├─ invoke_signed (CPI authority PDA signs)
│   └─ dWallet Program: approve_message
│       ├── Verifies caller_program is executable
│       ├── Verifies cpi_authority = PDA(["__ika_cpi_authority"],
caller_program)
│       ├── Verifies dwallet.authority == cpi_authority
│       └─ Creates MessageApproval PDA

```

The `DWalletContext` handles all of this – building the instruction data, assembling accounts, and calling `invoke_signed` with the correct seeds.

Client-Side Account Assembly

When constructing the transaction client-side, you need to know whether this vote will trigger quorum. If it will, include the extra 5 accounts:

```

let mut accounts = vec![
    AccountMeta::new(proposal_pda, false),
    AccountMeta::new(vote_record_pda, false),
    AccountMeta::new_readonly(voter.pubkey(), true),
    AccountMeta::new(payer.pubkey(), true),
    AccountMeta::new_readonly(system_program::id(), false),
];

// Include CPI accounts if this vote reaches quorum
if current_yes_votes + 1 >= quorum {
    accounts.extend_from_slice(&[
        AccountMeta::new(message_approval_pda, false),
        AccountMeta::new_readonly(dwallet_pda, false),
        AccountMeta::new_readonly(voting_program_id, false),
        AccountMeta::new_readonly(cpi_authority, false),
        AccountMeta::new_readonly(dwallet_program_id, false),
    ]);
}

```

Next Step

With voting and approval working, the next chapter shows how to [verify the resulting signature](#).

Verify Signature

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Reading the Signature

After the Ika network signs, the `MessageApproval` account is updated with the signature. You can read it from any client.

MessageApproval Account Layout

Offset	Field	Size	Description
0	discriminator	1	14
1	version	1	1
2	dwallet	32	dWallet pubkey
34	message_hash	32	Message hash that was signed
66	user_pubkey	32	User public key
98	signature_scheme	1	Ed25519(0), Secp256k1(1), Secp256r1(2)
99	caller_program	32	Program that approved
131	cpi_authority	32	CPI authority PDA
163	(internal fields)	...	...
139	status	1	Pending(0) or Signed(1)
140	signature_len	2	Signature byte count (LE u16)
142	signature	128	Signature bytes (padded)

Total: 287 bytes (2 + 285).

Polling for Signature Completion

```
use solana_rpc_client::rpc_client::RpcClient;

fn wait_for_signature(client: &RpcClient, message_approval: &Pubkey) -> Vec<u8>
{
    loop {
        let data = client.get_account(message_approval).unwrap().data;

        let status = data[139];
        if status == 1 { // Signed
            let sig_len = u16::from_le_bytes(
                data[140..142].try_into().unwrap()
            ) as usize;
            return data[142..142 + sig_len].to_vec();
        }

        std::thread::sleep(Duration::from_millis(500));
    }
}
```

Signature Verification

The signature can be verified against the dWallet's public key using standard cryptographic libraries. The verification algorithm depends on the signature scheme:

```
// Ed25519 verification example
use ed25519_dalek::{Signature, VerifyingKey};

let verifying_key = VerifyingKey::from_bytes(&dwallet_public_key)?;
let signature = Signature::from_bytes(&signature_bytes)?;
verifying_key.verify_strict(&message_hash, &signature)?;
```

For Secp256k1 (Bitcoin/Ethereum):

```
use secp256k1::{Message, PublicKey, Secp256k1, ecdsa::Signature};

let secp = Secp256k1::verification_only();
let pubkey = PublicKey::from_slice(&dwallet_public_key)?;
let message = Message::from_digest(message_hash);
let signature = Signature::from_compact(&signature_bytes)?;
secp.verify_ecdsa(&message, &signature, &pubkey)?;
```

E2E Flow Summary

The complete lifecycle from the E2E demo:

- | | |
|---|-------------------------------|
| 1. Create dWallet (CommitDWallet) | → dWallet PDA created |
| 2. Transfer authority to CPI PDA | → dWallet.authority = CPI PDA |
| 3. Create proposal (message_hash, quorum=3) | → Proposal PDA created |
| 4. Vote 1: Alice votes YES
yes_votes=1 | → VoteRecord created, |
| 5. Vote 2: Bob votes YES
yes_votes=2 | → VoteRecord created, |
| 6. Vote 3: Charlie votes YES (triggers quorum)
(Pending) | → MessageApproval created |
| 7. NOA signs and commits
(Signed) | → MessageApproval updated |
| 8. Read signature from MessageApproval
signature | → 64-byte Ed25519 |

Next Step

The next chapter covers [testing](#) the voting program at different levels.

Testing

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The voting example includes tests at two levels: Mollusk (instruction-level) and E2E (full lifecycle).

Mollusk Tests

Mollusk tests verify individual instructions in isolation. No validator needed.

```

use mollusk_svm::Mollusk;
use solana_instruction::Instruction;

fn setup() -> (Mollusk, Pubkey) {
    let program_id = Pubkey::new_unique();
    let mollusk = Mollusk::new(&program_id, PROGRAM_PATH);
    (mollusk, program_id)
}

#[test]
fn test_create_proposal_success() {
    let (mollusk, program_id) = setup();
    let creator = Pubkey::new_unique();
    let proposal_id = [0x01u8; 32];

    let (proposal_pda, proposal_bump) =
        Pubkey::find_program_address(&[b"proposal", &proposal_id],
&program_id);

    let ix = build_create_proposal_ix(/* ... */);
    let result = mollusk.process_instruction(&ix, &accounts);

    assert!(result.program_result.is_ok());
    let prop_data = &result.resulting_accounts[0].1.data;
    assert_eq!(prop_data[0], 1); // discriminator
    assert_eq!(read_u32(prop_data, 171), 3); // quorum
}

```

What to Test with Mollusk

Test	What It Verifies
test_create_proposal_success	Proposal PDA created with correct fields
test_create_proposal_already_exists	Fails when proposal account is non-empty
test_cast_vote_yes_success	Vote recorded, yes_votes incremented
test_cast_vote_no_success	No vote recorded, no_votes incremented
test_cast_vote_double_vote_fails	Second vote by same voter fails
test_cast_vote_closed_proposal_fails	Voting on approved proposal fails

Mollusk is fast (no network, no runtime startup) and tests instruction logic in isolation. However, it cannot test the CPI path (quorum triggering `approve_message`) because it runs a single program.

E2E Tests

The E2E demo runs the full lifecycle against Solana devnet and the pre-alpha dWallet gRPC service at `pre-alpha-dev-1.ika.ika-network.net:443`:

```
cargo run -p e2e-voting -- <DWALLET_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

The E2E test performs all 7 steps:

1. Wait for program state initialization (DWalletCoordinator + NEK)
2. Create dWallet via gRPC DKG + `CommitDWallet`
3. Transfer authority to voting program's CPI PDA
4. Create a voting proposal (quorum = 3)
5. Cast 3 YES votes (last triggers `approve_message` CPI)
6. Verify MessageApproval PDA exists with status = Pending
7. Presign + Sign via gRPC and verify signature on-chain

Assertions

The E2E test verifies:

- dWallet authority matches CPI PDA after transfer
- Proposal `yes_votes` = 3 and `status` = Approved after quorum
- MessageApproval exists with correct `dwallet` and `message_hash`
- Signature is committed and readable

See `chains/solana/examples/voting/e2e/src/main.rs` for the full implementation.

Running Tests

```
# Mollusk tests (fast, no validator)
cargo test -p ika-example-voting

# E2E (runs against devnet)
cd chains/solana/examples/voting/e2e
cargo run -- <DWALLET_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

dWallet Accounts

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

A dWallet is an on-chain account that represents a distributed signing key. It is created through Distributed Key Generation (DKG) and stored as a PDA owned by the dWallet program.

DWallet Account Layout

```
DWallet PDA:
  Seeds:  ["dwallet", chunks_of(curve_u16_le || public_key)]
  Program: DWALLET_PROGRAM_ID
```

The curve is stored as a `u16` (2 bytes, little-endian) concatenated with the raw public key into a single buffer, which is then split into 32-byte chunks (Solana’s `MAX_SEED_LEN`) and each chunk is passed as its own PDA seed. This is lossless and curve-agnostic – `find_program_address` accepts up to `MAX_SEEDS = 16` total seeds, so different pubkey lengths simply produce different chunk counts:

| pubkey | payload (`curve_u16_le || pk`) | chunks | |—|—|—| | 32 bytes (Ed25519 / Curve25519 / Ristretto) | 34 bytes | [32, 2] | | 33 bytes (compressed Secp256k1 / Secp256r1) | 35 bytes | [32, 3] | | 65 bytes (uncompressed SEC1) | 67 bytes | [32, 32, 3] |

Offset	Field	Size	Description
0	discriminator	1	2

Offset	Field	Size	Description
1	version	1	1
2	authority	32	Who can approve messages (user or CPI PDA)
34	curve	2	Curve type (u16 LE): 0=Secp256k1, 1=Secp256r1, 2=Curve25519, 3=Ristretto
36	state	1	0=DKGInProgress, 1=Active, 2=Frozen
37	public_key_len	1	Actual public key length (32 or 33)
38	public_key	65	dWallet public key (padded to 65 bytes)
103	created_epoch	8	Epoch when this dWallet was created (LE u64)
111	noa_public_key	32	NOA Ed25519 public key used during DKG
143	is_imported	1	Whether the key was imported (0=standard DKG, 1=imported)
144	bump	1	PDA bump seed
145	_reserved	8	Reserved for future use

Total: 153 bytes (2 header + 151 data)

Attestation data (DKG output, proofs, etc.) is stored in separate variable-size `dWalletAttestation` PDAs rooted from this dWallet's seed hierarchy, not inline in the dWallet account.

The `authority` field determines who can call `approve_message` for this dWallet:

- A **user pubkey** – the user signs the `approve_message` instruction directly
- A **CPI authority PDA** – a program controls the dWallet via CPI

DWalletAttestation Account

Variable-size PDA storing BCS-serialized versioned attestation data + NOA Ed25519 signature. One attestation PDA per type per dWallet.

Account layout: `[discriminator(1), version(1), noa_signature(64), bump(1), attestation_data...]`

Discriminator: 15 **Header:** 67 bytes (1 + 1 + 64 + 1), followed by variable-length attestation data.

Multiple PDA seed patterns depending on the type:

Type	Seeds
DKG	["dwallet", chunks..., "attestation"]
MakePublic	["dwallet", chunks..., "public_user_share"]
ReEncrypt	["dwallet", chunks..., "encrypted_user_share", &enc_key, "attestation"]
FutureSign	["dwallet", chunks..., "partial_user_sig", &scheme_u16_le, &msg_digest, [&meta_digest], "attestation"]

Creating a dWallet

dWallets are created through the gRPC API, not directly on-chain. The flow:

1. User sends a `DKG` request via gRPC with their key share
2. The Ika network runs the 2PC-MPC DKG protocol
3. The NOA calls `CommitDWallet` on-chain to create the dWallet account and its DKG attestation PDA
4. The dWallet's authority is set to the user

```
// Client-side: request DKG via gRPC
let request = DWalletRequest::DKG {
  dwallet_network_encryption_public_key: nek_bytes,
  curve: DWalletCurve::Secp256k1,
  centralized_public_key_share_and_proof: user_share,
  // Zero-trust mode. Use UserSecretKeyShare::Public { .. } for trust-
  minimized.
  user_secret_key_share: UserSecretKeyShare::Encrypted {
    encrypted_centralized_secret_share_and_proof: encrypted_share,
    encryption_key: enc_key,
    signer_public_key: signer_pk,
  },
  user_public_output: user_output,
  // Set to Some(SignDuringDKGRequest { .. }) to atomically sign a
  // message during DKG. `None` for plain DKG.
  sign_during_dkg_request: None,
};
```

Transferring Authority

To give a program control over a dWallet, transfer its authority to the program's CPI authority PDA:

```
// Derive the CPI authority PDA for your program
let (cpi_authority, _) = Pubkey::find_program_address(
    &[b"__ika_cpi_authority"],
    &your_program_id,
);

// TransferOwnership instruction (called by current authority)
let ix = Instruction::new_with_bytes(
    dwallet_program_id,
    &transfer_data, // [IX_TRANSFER_OWNERSHIP, new_authority(32)]
    vec![
        AccountMeta::new_readonly(current_authority, true), // signer
        AccountMeta::new(dwallet_pda, false),                // writable
    ],
);
```

After transfer, the dWallet's `authority` field equals the CPI authority PDA, and only the owning program can approve messages.

Via CPI (Program-to-Program Transfer)

If a program already controls a dWallet, it can transfer authority to another program's CPI PDA:

```
let ctx = DWalletContext {
    dwallet_program,
    cpi_authority,
    caller_program,
    cpi_authority_bump,
};

ctx.transfer_dwallet(dwallet, new_authority)?;
```

Supported Curves

Curve	ID (u16)	Key Size	Chains
Secp256k1	0	33 bytes (compressed)	Bitcoin, Ethereum, BSC
Secp256r1	1	33 bytes (compressed)	WebAuthn, Apple Secure Enclave
Curve25519	2	32 bytes	Solana, Sui, general Ed25519
Ristretto	3	32 bytes	Substrate, Polkadot

Reading dWallet Data Off-Chain

The `ika-solana-sdk-types` crate provides PDA derivation helpers:

```
use ika_sdk_types::pda::*;  
  
let (system_state, _) = find_system_state_address(&program_id);  
let (validator, _) = find_validator_address(&program_id, &identity);  
let (validator_list, _) = find_validator_list_address(&program_id);
```


Message Approval

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

Message approval is the core mechanism for requesting signatures from the Ika network. When you call `approve_message`, it creates a `MessageApproval` PDA on-chain. The network detects this account and produces a signature.

MessageApproval Account

MessageApproval PDA:

```
Seeds: ["dwallet", chunks..., "message_approval", &scheme_u16_le,
&message_digest, [&message_metadata_digest]]
Program: DWALLET_PROGRAM_ID
Total: 312 bytes (2 header + 310 data)
```

The PDA is rooted from the parent dWallet's `curve_u16_le || public_key` chunks (same hierarchy as all dWallet-derived PDAs). The `message_metadata_digest` seed is only included when non-zero.

The `message_digest` must be the **keccak256** hash of the message you want signed:

```
let message_digest = solana_sdk::keccak::hash(message).to_bytes();
```

```
import { keccak_256 } from "@noble/hashes/sha3.js";
const messageDigest = keccak_256(message);
```

This is consistent across all examples, the mock, and the gRPC service. Using any other hash function will result in a PDA mismatch when the network tries to commit the signature on-chain.

Offset	Field	Size	Description
0	discriminator	1	14
1	version	1	1
2	dwallet	32	dWallet account pubkey
34	message_digest	32	Keccak-256 digest of the message to sign
66	message_metadata_digest	32	Keccak-256 digest of message metadata (zero if none)
98	approver	32	dWallet authority who authorized the signing
130	user_pubkey	32	Public key authorized to call gRPC Sign
162	signature_scheme	2	DWalletSignatureScheme (u16 LE)
164	epoch	8	Epoch when the approval was created (LE u64)
172	status	1	Pending(0) or Signed(1)
173	signature_len	2	Length of the signature (LE u16)
175	signature	128	Signature bytes (padded)
303	bump	1	PDA bump seed
304	_reserved	8	Reserved for future use

Note: `signature_scheme` is now `[u8; 2]` (u16 LE) encoding a `DWalletSignatureScheme` value (0-6), not a single-byte `SignatureScheme`. The field `message_hash` has been renamed to `message_digest`, and `message_metadata_digest` is new.

Approval Flow

Direct Approval (User Signer)

When the dWallet's authority is a user wallet:

```
User signs approve_message instruction
-> dWallet program verifies user == dwallet.authority
-> Creates MessageApproval PDA (status = Pending)
```

CPI Approval (Program Signer)

When the dWallet's authority is a CPI authority PDA:

```
Your program calls DWalletContext::approve_message
-> invoke_signed with CPI authority seeds
-> dWallet program verifies:
  - caller_program is executable
  - cpi_authority == PDA(["__ika_cpi_authority"], caller_program)
  - dwallet.authority == cpi_authority
-> Creates MessageApproval PDA (status = Pending)
```

approve_message Instruction

Discriminator: 8

The first account is the `DWalletCoordinator` PDA (used to read the current epoch).

Instruction Data:

Offset	Field	Size
0	discriminator	1
1	bump	1
2	message_digest	32
34	message_metadata_digest	32
66	user_pubkey	32
98	signature_scheme	2

Accounts (CPI path):

#	Account	W	S	Description
0	coordinator	no	no	DWalletCoordinator PDA (for epoch)
1	message_approval	yes	no	MessageApproval PDA (must be empty)
2	dwallet	no	no	dWallet account
3	caller_program	no	no	Calling program (executable)
4	cpi_authority	no	yes	CPI authority PDA (signed via invoke_signed)
5	payer	yes	yes	Rent payer
6	system_program	no	no	System program

Signature Lifecycle

1. **Pending:** Your program calls `approve_message` -> `MessageApproval` created, `status = 0`, `signature_len = 0`
2. **gRPC Sign:** You send a `sign` request via gRPC with `ApprovalProof` referencing the on-chain approval. The network returns the 64-byte signature directly and commits it on-chain via `CommitSignature`.
3. **Signed:** `status = 1`, signature bytes written, readable by anyone.

Your program calls `approve_message` (CPI)

- > `MessageApproval` PDA created (`status = Pending`)
- > You send gRPC `Sign` request with `ApprovalProof`
- > Network signs and returns signature via gRPC
- > Network calls `CommitSignature` on-chain
- > `status = Signed`, signature available

The signature is available both from the gRPC response and on-chain in the `MessageApproval` account.

CommitSignature Instruction

Called by the NOA to write the signature into the `MessageApproval` account (or a `PartialUserSignature` account – dispatches by the target account's discriminator).

Discriminator: 43

Instruction Data:

Offset	Field	Size
0	discriminator	1
1	signature_len	2
3	signature	128

Accounts:

#	Account	W	S	Description
0	target_account	yes	no	<code>MessageApproval</code> or <code>PartialUserSignature</code> PDA
1	nek	no	no	<code>NetworkEncryptionKey</code> PDA
2	noa	no	yes	NOA signer

Reading the Signature

```
let data = client.get_account(&message_approval_pda)?.data;  
  
let status = data[172];  
if status == 1 {  
    let sig_len = u16::from_le_bytes(data[173..175].try_into().unwrap()) as  
    usize;  
    let signature = &data[175..175 + sig_len];  
    // Use the signature  
}
```

Idempotency

The same (dwallet_root, scheme, message_digest, message_metadata_digest) tuple always derives the same MessageApproval PDA. Attempting to create a MessageApproval that already exists will fail (the account is non-empty). This prevents duplicate signing requests.

CPI Framework

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

DWalletContext

The CPI SDK is available for four Solana frameworks:

Crate	Framework	Account type
ika-dwallet-pinocchio	Pinocchio	&AccountView
ika-dwallet-native	solana-program	&AccountInfo<'info>
ika-dwallet-anchor	Anchor v1	AccountInfo<'info>
ika-dwallet-quasar	Quasar	&AccountView (via .to_account_view())

All four provide an identical `DWalletContext` with the same methods and wire format.

```
use ika_dwallet_pinocchio::DWalletContext; // or _anchor, _native, _quasar

let ctx = DWalletContext {
    dwallet_program: &dwallet_program_account,
    cpi_authority: &cpi_authority_account,
    caller_program: &my_program_account,
    cpi_authority_bump: bump,
};
```

Field	Type	Description
dwallet_program	&AccountView	The dWallet program account
cpi_authority	&AccountView	Your program’s CPI authority PDA

Field	Type	Description
caller_program	&AccountView	Your program's account (must be executable)
api_authority_bump	u8	Bump seed for the CPI authority PDA

CPI Authority PDA

Every program derives its CPI authority from a single seed:

```
pub const CPI_AUTHORITY_SEED: &[u8] = b"__ika_cpi_authority";

// Derivation:
let (cpi_authority, bump) = Address::find_program_address(
    &[CPI_AUTHORITY_SEED],
    &your_program_id,
);
```

The dWallet program verifies this derivation during CPI calls.

Available Methods

approve_message

Creates a `MessageApproval` PDA requesting a signature. The first account is the `DWalletCoordinator` PDA (used to read the current epoch).

```
ctx.approve_message(
    coordinator,          // readonly -- DWalletCoordinator PDA (for epoch)
    message_approval,     // writable, empty -- PDA to create
    dwallet,              // readonly -- the dWallet account
    payer,                // writable, signer -- rent payer
    system_program,       // readonly -- system program
    message_digest,       // [u8; 32] -- keccak256 hash of message
    message_metadata_digest, // [u8; 32] -- keccak256 hash of metadata (zero if
    none)
    user_pubkey,          // [u8; 32] -- user public key
    signature_scheme,     // u16 -- DWalletSignatureScheme value (0-6)
    bump,                 // u8 -- MessageApproval PDA bump
)?;
```

CPI instruction data: `[8, bump, message_digest(32), message_metadata_digest(32), user_pubkey(32), signature_scheme(2)] = 100 bytes.`

CPI accounts:

#	Account	W	S
0	coordinator	no	no
1	message_approval	yes	no
2	dwallet	no	no
3	caller_program	no	no
4	cpi_authority	no	yes
5	payer	yes	yes
6	system_program	no	no

transfer_dwallet

Transfers dWallet authority to a new pubkey.

```
ctx.transfer_dwallet(  
    dwallet,          // writable -- the dWallet account  
    new_authority,    // [u8; 32] -- new authority pubkey  
)?;
```

CPI instruction data: [24, new_authority(32)] = 33 bytes.

CPI accounts:

#	Account	W	S
0	caller_program	no	no
1	cpi_authority	no	yes
2	dwallet	yes	no

transfer_future_sign

Transfers the completion authority of a `PartialUserSignature`.

```
ctx.transfer_future_sign(  
    partial_user_sig,          // writable -- partial signature account  
    new_completion_authority, // [u8; 32] -- new authority pubkey  
)?;
```

CPI instruction data: [42, new_completion_authority(32)] = 33 bytes.

CPI accounts:

#	Account	W	S
0	partial_user_sig	yes	no
1	caller_program	no	no
2	cpi_authority	no	yes

Signing Mechanism

All CPI methods use `invoke_signed` with the CPI authority seeds:

```
let bump_byte = [self.cpi_authority_bump];
let signer_seeds: [Seed; 2] = [
    Seed::from(CPI_AUTHORITY_SEED),
    Seed::from(&bump_byte),
];
let signer = Signer::from(&signer_seeds);

invoke_signed(&instruction, &accounts, &[signer])
```

The dWallet program verifies:

1. `caller_program` is executable
2. `cpi_authority` matches `PDA(["__ika_cpi_authority"], caller_program)`
3. `dwallet.authority == cpi_authority` (for `approve_message` and `transfer_dwallet`)

Instruction Discriminators

Instruction	Discriminator
<code>approve_message</code>	8
<code>transfer_ownership</code>	24
<code>commit_network_dkg</code>	28
<code>commit_network_key_reconfiguration</code>	30
<code>commit_dwallet</code>	31
<code>commit_future_sign</code>	33
<code>commit_encrypted_user_secret_key_share</code>	34
<code>commit_public_user_secret_key_share</code>	35
<code>transfer_future_sign</code>	42
<code>commit_signature</code>	43

Gas Deposits

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

GasDeposit Account

Every user has a `GasDeposit` PDA that holds IKA balance (for dWallet operation fees) and SOL balance (for NOA write-back transaction costs).

GasDeposit PDA:

Seeds: ["gas_deposit", user_pubkey]
Program: DWALLET_PROGRAM_ID
Total: 139 bytes (2 header + 137 data)
Discriminator: 4

Offset	Field	Size	Description
0	discriminator	1	4
1	version	1	1
2	user_pubkey	32	Ed25519 public key for gRPC authentication
34	ika_balance	8	Available IKA balance (LE u64)
42	sol_balance	8	Available SOL balance in lamports (LE u64)
50	total_ika_deposited	8	Lifetime IKA deposited (LE u64)
58	total_ika_consumed	8	Lifetime IKA consumed (LE u64)
66	total_sol_deposited	8	Lifetime SOL deposited (LE u64)
74	total_sol_consumed	8	Lifetime SOL consumed (LE u64)
82	pending_ika_withdrawal	8	Pending IKA withdrawal amount (LE u64)

Offset	Field	Size	Description
90	pending_sol_withdrawal	8	Pending SOL withdrawal amount (LE u64)
98	withdrawal_epoch	8	Epoch when pending withdrawal becomes available (LE u64, 0=none)
106	last_settlement_epoch	8	Epoch of last gas settlement (LE u64)
114	created_at_epoch	8	Epoch when deposit was created (LE u64)
122	bump	1	PDA bump seed
123	_reserved	16	Reserved for future use

Gas Deposit Instructions

Instruction	Discriminator	Description
CreateDeposit	36	Create a new GasDeposit PDA for a user
TopUp	37	Add IKA or SOL to an existing deposit
SettleGas	38	NOA settles consumed gas (periodic)
RequestWithdraw	44	Request withdrawal (sets pending amount + epoch)
Withdraw	45	Complete withdrawal after epoch delay

Rent Costs by Account Type

The dWallet program uses a simplified rent formula:

```
fn minimum_balance(data_len: usize) -> u64 {  
    (data_len as u64 + 128) * 6960  
}
```

This approximation of the Solana rent-exempt minimum is used for all PDA creation.

Account	Size (bytes)	Approximate Rent (lamports)
DWallet	153	~1,955,280
DWalletAttestation	67 + data	varies
MessageApproval	312	~3,062,400
PartialUserSignature	570	~4,858,080

Account	Size (bytes)	Approximate Rent (lamports)
EncryptedUserSecretKeyShare	148	~1,920,480
GasDeposit	139	~1,858,320
DWalletCoordinator	116	~1,698,240
Proposal (voting example)	195	~2,248,080
VoteRecord (voting example)	69	~1,371,480

Payer Account

Every instruction that creates a PDA requires a `payer` account:

- Must be writable and signer
- Must have sufficient lamports to cover rent
- Is debited via `CreateAccount` system instruction

Future: Production Gas Model

In production, the Ika network will have a gas model for signing operations. This may include:

- Presign allocation fees
- Signing operation fees
- Staking requirements for validators

The exact model is not finalized.

SubmitTransaction

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

`SubmitTransaction` is the primary gRPC RPC for all dWallet operations. It accepts a `UserSignedRequest` and returns a `TransactionResponse`.

The request type is determined by the `DWalletRequest` enum variant inside the BCS-serialized payload. This means a single RPC endpoint handles DKG, signing, presigning, and other operations.

Service Definition

```
service DWalletService {  
  rpc SubmitTransaction(UserSignedRequest) returns (TransactionResponse);  
  rpc GetPresigns(GetPresignsRequest) returns (GetPresignsResponse);  
  rpc GetPresignsForDWallet(GetPresignsForDWalletRequest) returns  
  (GetPresignsResponse);  
}
```

UserSignedRequest

All mutation requests are wrapped in `UserSignedRequest`:

```
message UserSignedRequest {  
    bytes user_signature = 1;      // BCS-serialized UserSignature enum  
    bytes signed_request_data = 2; // BCS-serialized SignedRequestData  
}
```

Field	Type	Description
user_signature	bytes	BCS-serialized <code>UserSignature</code> enum (signature + public key + scheme)
signed_request_data	bytes	BCS-serialized <code>SignedRequestData</code> (the signed payload)

The `user_signature` covers the `signed_request_data` bytes – validators independently verify the signature.

Authentication

The `UserSignature` enum is self-contained: it carries both the signature bytes and the public key bytes, with the variant determining the scheme:

```
pub enum UserSignature {  
    Ed25519 {  
        signature: Vec<u8>, // 64 bytes  
        public_key: Vec<u8>, // 32 bytes  
    },  
    Secp256k1 {  
        signature: Vec<u8>, // 64 bytes  
        public_key: Vec<u8>, // 33 bytes (compressed)  
    },  
    Secp256r1 {  
        signature: Vec<u8>, // 64 bytes  
        public_key: Vec<u8>, // 33 bytes (compressed)  
    },  
}
```

Signed Payload

The `SignedRequestData` struct contains the operation to perform:

```
pub struct SignedRequestData {  
    pub session_identifier_preimage: [u8; 32],  
    pub epoch: u64,  
    pub chain_id: ChainId,  
    pub intended_chain_sender: Vec<u8>,  
    pub request: DWalletRequest,  
}
```

Field	Description
session_identifier_preimage	Random 32 bytes (uniqueness nonce)
epoch	Current Ika epoch (prevents cross-epoch replay)
chain_id	Solana OR Sui
intended_chain_sender	User's address on the target chain
request	The DWalletRequest enum variant

TransactionResponse

```
message TransactionResponse {  
    bytes response_data = 1; // BCS-serialized TransactionResponseData  
}
```

Deserialize `response_data` into `TransactionResponseData` to get the result:

```
pub enum TransactionResponseData {  
    Signature { signature: Vec<u8> },  
    Attestation(NetworkSignedAttestation),  
    Error { message: String },  
}
```

Three variants only – presigns now flow through `Attestation(NetworkSignedAttestation)` .

Client Usage

```
use ika_grpc::d_wallet_service_client::DWalletServiceClient;
use ika_grpc::UserSignedRequest;

let mut client = DWalletServiceClient::connect(
    "https://pre-alpha-dev-1.ika.ika-network.net:443"
).await?;

let resp = client.submit_transaction(UserSignedRequest {
    user_signature: bcs::to_bytes(&user_sig)?,
    signed_request_data: bcs::to_bytes(&signed_data)?,
}).await?;

let tx_response = resp.into_inner();
let result: TransactionResponseData =
    bcs::from_bytes(&tx_response.response_data)?;
```

Query RPCs

GetPresigns

Get all global presigns for a user.

```
message GetPresignsRequest {
    bytes user_pubkey = 1;
}
```

GetPresignsForDWallet

Get all presigns for a specific dWallet.

```
message GetPresignsForDWalletRequest {
    bytes user_pubkey = 1;
    bytes dwallet_id = 2;
}
```


GetPresignsResponse

```
message GetPresignsResponse {  
  repeated PresignInfo presigns = 1;  
}  
  
message PresignInfo {  
  bytes presign_id = 1;  
  bytes dwallet_id = 2;  
  uint32 curve = 3;  
  uint32 signature_scheme = 4;  
  uint64 epoch = 5;  
}
```

Request Types

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

DWalletRequest Enum

All operations are encoded as variants of the `DWalletRequest` enum, BCS-serialized inside `SignedRequestData.request`.

```
pub enum DWalletRequest {
    DKG { ... },
    Sign { ... },
    ImportedKeySign { ... },
    Presign { ... },
    PresignForDWallet { ... },
    ImportedKeyVerification { ... },
    ReEncryptShare { ... },
    MakeSharePublic { ... },
    FutureSign { ... },
    SignWithPartialUserSig { ... },
    ImportedKeySignWithPartialUserSig { ... },
}
```

Mock Support

All request types are implemented and tested end-to-end (see `protocols-e2e` example).

Request	Status	Notes
DKG	Supported	All 4 curves (Secp256k1, Secp2 Curve25519, Ristretto). Encrypt share mode. Auto-commits dV and transfers authority to

Request	Status	Notes
		<code>intended_chain_sender</code> . Ristretto Schnorrkel keypairs.
Sign	Supported	7 signature schemes (ECDSA, Schnorrkel, and scalar variants). Returns <code>signature_scheme</code> from on-chain <code>MessageApproval</code> . Supports 1 cross-chain digest computation EVM, DoubleSHA256 for Bitcoin.
ImportedKeySign	Supported	Same as Sign but for imported-key dWallets.
Presign	Supported	Returns attestation with <code>presignature_algorithm</code> (not <code>signature_scheme</code>).
PresignForDWallet	Supported	Same as Presign. Uses <code>dwallet_id</code> (not <code>dwallet_id</code>). Includes <code>dwallet_attestation</code> for verification.
ImportedKeyVerification	Supported	Creates an imported-key dWallet. Returns <code>UserSecretKeyShare</code> (Encryption key share).
ReEncryptShare	Supported	Re-encrypts the user's secret key with a new encryption key. Returns <code>VersionedEncryptedUserKeyShare</code> .
MakeSharePublic	Supported	Converts an encrypted share to a public key. Returns <code>VersionedPublicUserKeyShare</code> .
FutureSign	Supported	Two-step conditional signing (partial user signature that can be completed later via <code>SignWithPartialUserSignature</code>). Returns <code>VersionedPartialUserSignature</code> .
SignWithPartialUserSignature	Supported	Two-step conditional signing (completes a partial signature from <code>FutureSign</code>).
ImportedKeySignWithPartialUserSignature	Supported	Same as <code>SignWithPartialUserSignature</code> for imported-key dWallets.

Supported Curves

Curve	DKG	Presign	Notes
Secp256k1	Yes	Yes	Bitcoin, Ethereum
Secp256r1	Yes	Yes	WebAuthn, secure enclaves
Curve25519	Yes	Yes	Solana, Sui (Ed25519)
Ristretto	Yes	Yes	Substrate, Polkadot (Schnorrkel)

DKG

Create a new dWallet via Distributed Key Generation. The `user_secret_key_share` field selects between **zero-trust** mode (encrypted user share) and **trust-minimized** mode (public user share) – mirrors Sui move `UserSecretKeyShareEventType` .

```
DWalletRequest::DKG {
  dwallet_network_encryption_public_key: Vec<u8>,
  curve: DWalletCurve,
  centralized_public_key_share_and_proof: Vec<u8>,
  user_secret_key_share: UserSecretKeyShare,
  user_public_output: Vec<u8>,
  sign_during_dkg_request: Option<SignDuringDKGRequest>,
}

pub enum UserSecretKeyShare {
  /// Zero-trust mode.
  Encrypted {
    encrypted_centralized_secret_share_and_proof: Vec<u8>,
    encryption_key: Vec<u8>,
    signer_public_key: Vec<u8>, // Ed25519, signs the public output to
    prove ownership
  },
  /// Trust-minimized mode -- secret share revealed.
  Public {
    public_user_secret_key_share: Vec<u8>,
  },
}
```

Field	Description
<code>dwallet_network_encryption_public_key</code>	Network encryption key (from on-chain NEK account)
<code>curve</code>	Target curve (Secp256k1, Secp256r1, Curve25519, Ristretto)
<code>centralized_public_key_share_and_proof</code>	User's public key share + ZK proof
<code>user_secret_key_share</code>	Encrypted { ... } for zero-trust, Public { ... } for trust-minimized
<code>user_public_output</code>	User's DKG public output
<code>sign_during_dkg_request</code>	Optional – atomically sign a message during DKG (None for plain DKG)

Note: `signer_public_key` lives inside the `Encrypted` variant only. Trust-minimized mode has no secret to prove possession of.

Response: `TransactionResponseData::Attestation(NetworkSignedAttestation)` with the DKG output and NOA attestation. The `attestation_data` decodes to `VersionedDWalletDataAttestation` .

SignDuringDKGRequest

Optional payload attached to `DKG` to atomically sign a message during DKG.

```
pub struct SignDuringDKGRequest {  
    pub presign_session_identifier: Vec<u8>,  
    pub presign: Vec<u8>,  
    pub signature_scheme: DWalletSignatureScheme,  
    pub message: Vec<u8>,  
    pub message_metadata: Vec<u8>,  
    pub message_centralized_signature: Vec<u8>,  
}
```

Field	Description
<code>presign_session_identifier</code>	Presign session identifier (from a prior Presign response)
<code>presign</code>	Presign material
<code>signature_scheme</code>	<code>DWalletSignatureScheme</code> enum
<code>message</code>	Raw message bytes to sign
<code>message_metadata</code>	BCS-serialized per-scheme metadata (empty for most schemes)
<code>message_centralized_signature</code>	User's centralized-party partial signature

The curve is inherited from the parent DKG request.

Sign

Sign a message using an existing dWallet.

```
DWalletRequest::Sign {  
    message: Vec<u8>,  
    message_metadata: Vec<u8>,  
    presign_session_identifier: Vec<u8>,  
    message_centralized_signature: Vec<u8>,  
    dwallet_attestation: NetworkSignedAttestation,  
    approval_proof: ApprovalProof,  
}
```

Field	Description
<code>message</code>	Raw message bytes to sign
<code>message_metadata</code>	BCS-serialized per-scheme metadata (see <code>Blake2bMessageMetadata</code> , <code>SchnorrkelMessageMetadata</code>). Empty for most schemes.

Field	Description
<code>presign_session_identifier</code>	Session identifier of a previously allocated presign
<code>message_centralized_signature</code>	User's partial signature
<code>dwallet_attestation</code>	<code>NetworkSignedAttestation</code> from the DKG response (proves the dWallet exists)
<code>approval_proof</code>	On-chain proof of message approval

Note: `curve` and `signature_scheme` are no longer fields on `Sign` – validators derive the signature scheme from the on-chain `MessageApproval` and the curve from the `dwallet_attestation`.

Response: `TransactionResponseData::Signature` with the completed signature.

ImportedKeySign

Same as `Sign` but for imported-key dWallets. Validators additionally verify `is_imported_key == true` on the referenced dWallet.

```
DWalletRequest::ImportedKeySign {
    message: Vec<u8>,
    message_metadata: Vec<u8>,
    presign_session_identifier: Vec<u8>,
    message_centralized_signature: Vec<u8>,
    dwallet_attestation: NetworkSignedAttestation,
    approval_proof: ApprovalProof,
}
```

ApprovalProof

The approval proof ties the gRPC signing request to an on-chain `MessageApproval`:

```
pub enum ApprovalProof {
    Solana {
        transaction_signature: Vec<u8>, // Solana tx signature
        slot: u64,                       // Slot of the transaction
    },
    Sui {
        effects_certificate: Vec<u8>,    // Sui effects certificate
    },
}
```

Presign

Allocate a global presign (usable with any non-imported dWallet for the same `signature_algorithm`).

```
DWalletRequest::Presign {  
    dwallet_network_encryption_public_key: Vec<u8>,  
    curve: DWalletCurve,  
    signature_algorithm: DWalletSignatureAlgorithm,  
}
```

Field	Description
<code>dwallet_network_encryption_public_key</code>	Network encryption key
<code>curve</code>	Target curve
<code>signature_algorithm</code>	<code>DWalletSignatureAlgorithm</code> (<code>ECDSASecp256k1</code> , <code>ECDSASecp256r1</code> , <code>Taproot</code> , <code>EdDSA</code> , <code>Schnorrkel</code>)

Note: uses `signature_algorithm` (not `signature_scheme`). Presigns are per-algorithm, not per-scheme, because the hash function is applied at signing time.

Response: `TransactionResponseData::Attestation(NetworkSignedAttestation)`. The `attestation_data` decodes to `VersionedPresignDataAttestation`.

PresignForDWallet

Allocate a presign bound to a specific dWallet (required for imported ECDSA dWallets). Runs a full 2-round MPC presign protocol – significantly slower than global presigns.

```
DWalletRequest::PresignForDWallet {  
    dwallet_network_encryption_public_key: Vec<u8>,  
    dwallet_public_key: Vec<u8>,  
    curve: DWalletCurve,  
    signature_algorithm: DWalletSignatureAlgorithm,  
}
```

Field	Description
<code>dwallet_network_encryption_public_key</code>	Network encryption key
<code>dwallet_public_key</code>	Public key of the target dWallet (not a dWallet ID)
<code>curve</code>	Target curve
<code>signature_algorithm</code>	<code>DWalletSignatureAlgorithm</code>

NetworkSignedAttestation

Common response / request payload for state-creating operations – carries a network-signed blob the user can either (a) submit on-chain to claim the result or (b) feed back to the network in a follow-up request (e.g. `SignWithPartialUserSig`).

```
pub struct NetworkSignedAttestation {  
    pub attestation_data: Vec<u8>, // BCS-serialized per-type versioned  
    attestation struct  
    pub network_signature: Vec<u8>, // Ed25519 signature from the NOA  
    pub network_pubkey: Vec<u8>, // NOA public key (matches active  
    NetworkEncryptionKey)  
    pub epoch: u64, // Epoch this attestation was produced  
in  
}
```

The `attestation_data` contains BCS-serialized bytes of a per-type versioned struct. The caller knows which type based on the originating request:

Request	Attestation Type
DKG / ImportedKeyVerification	VersionedDWalletDataAttestation
Presign / PresignForDWallet	VersionedPresignDataAttestation
FutureSign	VersionedPartialUserSignatureAttestation
ReEncryptShare	VersionedEncryptedUserKeyShareAttestation
MakeSharePublic	VersionedPublicUserKeyShareAttestation

ImportedKeyVerification

Verify an externally-generated key as a new dWallet (no DKG). Uses `UserSecretKeyShare` to select zero-trust or trust-minimized mode, same as DKG.

```
DWalletRequest::ImportedKeyVerification {  
    dwallet_network_encryption_public_key: Vec<u8>,  
    curve: DWalletCurve,  
    centralized_party_message: Vec<u8>,  
    user_secret_key_share: UserSecretKeyShare,  
    user_public_output: Vec<u8>,  
}
```

Field	Description
<code>dwallet_network_encryption_public_key</code>	Network encryption key
<code>curve</code>	Target curve
<code>centralized_party_message</code>	Centralized party verification message

Field	Description
user_secret_key_share	UserSecretKeyShare::Encrypted { ... } Or Public { ... }
user_public_output	User's public output

Response: TransactionResponseData::Attestation(NetworkSignedAttestation) . User submits the attestation on-chain to create the imported-key dWallet.

ReEncryptShare

Re-encrypt a dWallet's user secret share under a new encryption key (to transfer / grant access). Wire format defined; not yet implemented in mock.

```
DWalletRequest::ReEncryptShare {  
  dwallet_network_encryption_public_key: Vec<u8>,  
  dwallet_public_key: Vec<u8>,  
  dwallet_attestation: NetworkSignedAttestation,  
  encrypted_centralized_secret_share_and_proof: Vec<u8>,  
  encryption_key: Vec<u8>,  
}
```

Field	Description
dwallet_network_encryption_public_key	Network encryption key
dwallet_public_key	Public key of the target dWallet
dwallet_attestation	The dWallet's DKG attestation
encrypted_centralized_secret_share_and_proof	The re-encrypted share + proof
encryption_key	New encryption key

The previous share (the source) and the dWallet's public_output are looked up by validators from local state using dwallet_public_key .

Response: TransactionResponseData::Attestation(NetworkSignedAttestation) . The attestation_data decodes to VersionedEncryptedUserKeyShareAttestation .

MakeSharePublic

Transition a zero-trust dWallet to trust-minimized by revealing the user's secret key share. One-way. Wire format defined; not yet implemented in mock.

```
DWalletRequest::MakeSharePublic {  
    dwallet_public_key: Vec<u8>,  
    dwallet_attestation: NetworkSignedAttestation,  
    public_user_secret_key_share: Vec<u8>,  
}
```

Field	Description
dwallet_public_key	Public key of the target dWallet
dwallet_attestation	The dWallet's DKG attestation
public_user_secret_key_share	The revealed secret key share

Response: `TransactionResponseData::Attestation(NetworkSignedAttestation)`. The `attestation_data` decodes to `VersionedSignedPublicUserKeyShareAttestation`.

FutureSign

Step 1 of two-step conditional signing – produce a verified partial user signature without an approval proof. Consumes a presign. Wire format defined; not yet implemented in mock.

```
DWalletRequest::FutureSign {  
    dwallet_public_key: Vec<u8>,  
    presign_session_identifier: Vec<u8>,  
    message: Vec<u8>,  
    message_metadata: Vec<u8>,  
    message_centralized_signature: Vec<u8>,  
    signature_scheme: DWalletSignatureScheme,  
}
```

Field	Description
dwallet_public_key	Public key of the target dWallet
presign_session_identifier	Presign session identifier
message	Raw message bytes to sign
message_metadata	BCS-serialized per-scheme metadata (empty for most schemes)
message_centralized_signature	User's partial signature
signature_scheme	<code>DWalletSignatureScheme</code> – kept here since <code>FutureSign</code> has no approval proof to derive it from

Response: `TransactionResponseData::Attestation(NetworkSignedAttestation)` (the verified partial signature, ready to feed into `SignWithPartialUserSig`). The `attestation_data` decodes to `VersionedPartialUserSignatureAttestation`.

SignWithPartialUserSig

Step 2 of two-step conditional signing – complete the signature using the attestation returned by `FutureSign`. Requires an on-chain approval proof, just like `Sign`. Wire format defined; not yet implemented in mock.

```
DWalletRequest::SignWithPartialUserSig {  
    partial_user_signature_attestation: NetworkSignedAttestation,  
    dwallet_attestation: NetworkSignedAttestation,  
    approval_proof: ApprovalProof,  
}
```

Field	Description
<code>partial_user_signature_attestation</code>	Attestation from <code>FutureSign</code>
<code>dwallet_attestation</code>	The dWallet's DKG attestation
<code>approval_proof</code>	On-chain proof of message approval

Response: `TransactionResponseData::Signature`.

ImportedKeySignWithPartialUserSig

Imported-key variant of `SignWithPartialUserSig`. Validators additionally verify the referenced dWallet was created from an imported key. Wire format defined; not yet implemented in mock.

```
DWalletRequest::ImportedKeySignWithPartialUserSig {  
    partial_user_signature_attestation: NetworkSignedAttestation,  
    dwallet_attestation: NetworkSignedAttestation,  
    approval_proof: ApprovalProof,  
}
```

Cryptographic Parameter Enums

DWalletCurve

Variant	Value	Description
<code>Secp256k1</code>	0	Bitcoin, Ethereum
<code>Secp256r1</code>	1	WebAuthn, secure enclaves
<code>Curve25519</code>	2	Solana, Sui, Ed25519

Variant	Value	Description
Ristretto	3	Substrate, Polkadot

On-wire encoding: `u16` (LE in on-chain accounts, BCS-serialized for gRPC).

DWalletSignatureScheme

Combined (algorithm, hash) pair. Eliminates impossible combinations like `ECDSA + Merlin` at the type level. The on-wire encoding is `u16 (#[repr(u16)])`.

Variant	Index	Curve	Use For
EcdsaKeccak256	0	Secp256k1	Ethereum
EcdsaSha256	1	Secp256k1 / Secp256r1	Bitcoin (legacy) / WebAuthn
EcdsaDoubleSha256	2	Secp256k1	Bitcoin BIP143
TaprootSha256	3	Secp256k1	Bitcoin Taproot (BIP340)
EcdsaBlake2b256	4	Secp256k1	Zcash (personal/salt via <code>message_metadata</code>)
EddsaSha512	5	Curve25519	Ed25519 (Solana, Sui)
SchnorrkelMerlin	6	Ristretto	Substrate, Polkadot (sr25519)

Not every (curve, scheme) combination is valid. Validators reject invalid pairs (e.g. `Curve25519 + EcdsaKeccak256`, `Secp256r1 + Taproot`). Ordering: variants 0-4 are Secp256k1 (with 1 also usable on Secp256r1), variant 5 is Curve25519, variant 6 is Ristretto.

DWalletSignatureAlgorithm

Used by `Presign` and `PresignForDWallet` requests (presigns are per-algorithm, not per-scheme):

Variant	Value	Description
ECDSASecp256k1	0	ECDSA on Secp256k1
ECDSASecp256r1	1	ECDSA on Secp256r1
Taproot	2	Schnorr on Secp256k1
EdDSA	3	Ed25519 on Curve25519
Schnorrkel	4	sr25519 on Ristretto

Message Metadata

Some signature schemes require additional metadata, BCS-serialized and passed in the `message_metadata` field:

Blake2bMessageMetadata (for `EcdsaBlake2b256`):

```
pub struct Blake2bMessageMetadata {  
    pub personal: Vec<u8>, // BLAKE2b personalization (up to 16 bytes)  
    pub salt: Vec<u8>,    // BLAKE2b salt (up to 16 bytes, empty for most  
uses)  
}
```

Example (Zcash): `personal: b"ZcashSigHash\x00\x00\x00\x00"`, `salt: vec![]`.

SchnorrkelMessageMetadata (for `SchnorrkelMerlin`):

```
pub struct SchnorrkelMessageMetadata {  
    pub context: Vec<u8>, // Signing context (domain separator for Merlin  
transcript)  
}
```

Example (Substrate): `context: b"substrate"`. If empty, validators default to `b"substrate"`.

DWalletSignatureAlgorithm / DWalletHashScheme (internal)

The internal MPC stack still uses these granular enums. They are not on the wire – the gRPC adapter converts `DWalletSignatureScheme` to/from these at the validator boundary via `to_internal()` / `from_internal()`.

ChainId

Variant	Description
Solana	Solana blockchain
Sui	Sui blockchain

SignatureScheme (User Authentication)

Used in `UserSignature` for gRPC request authentication (not for dWallet signing):

Variant	Value	Key Size
Ed25519	0	32 bytes
Secp256k1	1	33 bytes

Variant	Value	Key Size
Secp256r1	2	33 bytes

Response Types

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

TransactionResponseData

The `SubmitTransaction` RPC returns a `TransactionResponse` containing BCS-serialized `TransactionResponseData`:

```
pub enum TransactionResponseData {  
    Signature { signature: Vec<u8> },  
    Attestation(NetworkSignedAttestation),  
    Error { message: String },  
}
```

Three variants only. Presigns are now NOA-signed and flow through `Attestation` – there is no separate `Presign` variant.

Response Variants

Signature

Returned for `Sign`, `ImportedKeySign`, `SignWithPartialUserSig`, and `ImportedKeySignWithPartialUserSig` requests.

```
TransactionResponseData::Signature {  
    signature: Vec<u8>, // The completed signature bytes  
}
```

Field	Type	Description
signature	Vec<u8>	The completed digital signature

The signature is always 64 bytes:

- **ECDSA (Secp256k1 / Secp256r1)**: 64 bytes (r || s)
- **Taproot (BIP340)**: 64 bytes (Schnorr signature)
- **EdDSA**: 64 bytes (Ed25519 signature)
- **Schnorrkel**: 64 bytes (sr25519 signature)

Attestation

Returned for all state-creating operations: `DKG`, `ImportedKeyVerification`, `Presign`, `PresignForDWallet`, `FutureSign`, `ReEncryptShare`, and `MakeSharePublic`.

`TransactionResponseData::Attestation(NetworkSignedAttestation)`

```
pub struct NetworkSignedAttestation {
    pub attestation_data: Vec<u8>,    // BCS-serialized per-type versioned
    attestation struct
    pub network_signature: Vec<u8>,    // NOA Ed25519 signature over
    attestation_data
    pub network_pubkey: Vec<u8>,      // NOA public key
    pub epoch: u64,                  // Epoch of the attestation
}
```

Field	Type	Description
attestation_data	Vec<u8>	BCS-serialized per-type versioned struct (see below)
network_signature	Vec<u8>	NOA's Ed25519 signature attesting to the output
network_pubkey	Vec<u8>	NOA's public key for verification
epoch	u64	Ika epoch when the attestation was produced

The `attestation_data` bytes decode to a per-type versioned struct based on the originating request:

Request	Attestation Type	Description
DKG / ImportedKeyVerification	VersionedDWalletDataAttestation	DKG output (public proofs,
Presign / PresignForDWallet	VersionedPresignDataAttestation	Presign session identifier data

Request	Attestation Type	Description
FutureSign	VersionedPartialUserSignatureAttestation	Verified partial signature
ReEncryptShare	VersionedEncryptedUserKeyShareAttestation	Re-encrypted share commitment
MakeSharePublic	VersionedPublicUserKeyShareAttestation	Public user key share commitment

Error

Returned when the operation fails.

```
TransactionResponseData::Error {  
    message: String, // Human-readable error description  
}
```

Always check for the `Error` variant before processing the response.

Per-Type Versioned Attestation Structs

Each operation type has its own versioned BCS enum. The same `(attestation_data, network_signature)` pair is stored on-chain (in the corresponding PDA) and returned via gRPC.

VersionedDWalletDataAttestation

For DKG and ImportedKeyVerification results.

```
pub enum VersionedDWalletDataAttestation {  
    V1(DWalletDataAttestationV1),  
}  
  
pub struct DWalletDataAttestationV1 {  
    pub session_identifier: [u8; 32],  
    pub intended_chain_sender: Vec<u8>,  
    pub curve: DWalletCurve,  
    pub public_key: Vec<u8>,  
    pub public_output: Vec<u8>,  
    pub is_imported_key: bool,  
    pub sign_during_dkg_signature: Option<Vec<u8>>,  
}
```

VersionedPresignDataAttestation

For Presign and PresignForDWallet results.

```
pub enum VersionedPresignDataAttestation {
    V1(PresignDataAttestationV1),
}

pub struct PresignDataAttestationV1 {
    pub session_identifier: [u8; 32],
    pub epoch: u64,
    pub presign_session_identifier: Vec<u8>,
    pub presign_data: Vec<u8>,
    pub curve: DWalletCurve,
    pub signature_algorithm: DWalletSignatureAlgorithm,
    pub dwallet_public_key: Option<Vec<u8>>, // None for global, Some for
dwallet-specific
    pub user_pubkey: Vec<u8>,
}
```

Note: signature_algorithm (not signature_scheme). dwallet_public_key (not dwallet_id).

VersionedPartialUserSignatureAttestation

For FutureSign results.

```
pub enum VersionedPartialUserSignatureAttestation {
    V1(PartialUserSignatureAttestationV1),
}

pub struct PartialUserSignatureAttestationV1 {
    pub session_identifier: [u8; 32],
    pub intended_chain_sender: Vec<u8>,
    pub dwallet_public_key: Vec<u8>,
    pub presign_session_identifier: Vec<u8>,
    pub message: Vec<u8>,
    pub signature_scheme: DWalletSignatureScheme,
}
```

VersionedEncryptedUserKeyShareAttestation

For ReEncryptShare results.

```
pub enum VersionedEncryptedUserKeyShareAttestation {
    V1(EncryptedUserKeyShareAttestationV1),
}

pub struct EncryptedUserKeyShareAttestationV1 {
    pub session_identifier: [u8; 32],
    pub intended_chain_sender: Vec<u8>,
    pub dwallet_public_key: Vec<u8>,
    pub encrypted_centralized_secret_share_and_proof: Vec<u8>,
}
```

VersionedPublicUserKeyShareAttestation

For MakeSharePublic results.

```
pub enum VersionedPublicUserKeyShareAttestation {
    V1(PublicUserKeyShareAttestationV1),
}

pub struct PublicUserKeyShareAttestationV1 {
    pub session_identifier: [u8; 32],
    pub intended_chain_sender: Vec<u8>,
    pub dwallet_public_key: Vec<u8>,
    pub public_user_secret_key_share: Vec<u8>,
}
```

Deserialization Example

```
use ika_dwallet_types::{TransactionResponseData, NetworkSignedAttestation};

let response = client.submit_transaction(request).await?;
let result: TransactionResponseData =
    bcs::from_bytes(&response.into_inner().response_data)?;

match result {
    TransactionResponseData::Signature { signature } => {
        println!("Got signature: {} bytes", signature.len());
    }
    TransactionResponseData::Attestation(NetworkSignedAttestation {
        attestation_data, network_signature, epoch, ..
    }) => {
        println!("Attestation: {} bytes, epoch {}", attestation_data.len(),
epoch);
        // Submit on-chain (e.g. CommitDWallet) or decode per-type struct
    }
    TransactionResponseData::Error { message } => {
        eprintln!("Error: {message}");
    }
}
```

PresignInfo (Query Response)

Returned by `GetPresigns` and `GetPresignsForDWallet`:

```
// Proto message
message PresignInfo {
  bytes presign_id = 1;
  bytes dwallet_id = 2;
  uint32 curve = 3;
  uint32 signature_scheme = 4;
  uint64 epoch = 5;
}
```

Field	Type	Description
presign_id	bytes	Unique presign identifier
dwallet_id	bytes	Associated dWallet (empty for global presigns)
curve	u32	Curve identifier
signature_scheme	u32	Signature scheme identifier
epoch	u64	Epoch when allocated

Mollusk Tests

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

Mollusk is the fastest way to test individual instructions in isolation. It runs a single instruction against pre-built account state – no validator, no network, no startup cost.

Mollusk is best for:

- Verifying instruction data parsing
- Checking signer and account validation
- Testing discriminator handling
- Validating PDA creation and field writes
- Testing error conditions (double votes, closed proposals, missing signers)

Mollusk **cannot** test CPI calls (e.g., quorum triggering `approve_message`), because it runs a single program in isolation.

Setup

```
[dev-dependencies]
mollusk-svm = "0.2"
solana-account = "2"
solana-instruction = "2"
solana-pubkey = "2"
```

```

use mollusk_svm::Mollusk;
use solana_account::Account;
use solana_instruction::{AccountMeta, Instruction};
use solana_pubkey::Pubkey;

const PROGRAM_PATH: &str = concat!(
    env!("CARGO_MANIFEST_DIR"),
    "/../../target/deploy/ika_example_voting"
);

fn setup() -> (Mollusk, Pubkey) {
    let program_id = Pubkey::new_unique();
    let mollusk = Mollusk::new(&program_id, PROGRAM_PATH);
    (mollusk, program_id)
}

```

Account Helpers

Pre-build account state for test inputs:

```

fn funded_account() -> Account {
    Account {
        lamports: 10_000_000_000,
        data: vec![],
        owner: SYSTEM_PROGRAM_ID,
        executable: false,
        rent_epoch: 0,
    }
}

fn program_account(owner: &Pubkey, data: Vec<u8>) -> Account {
    Account {
        lamports: ((data.len() as u64 + 128) * 6960).max(1),
        data,
        owner: *owner,
        executable: false,
        rent_epoch: 0,
    }
}

fn empty_account() -> Account {
    Account {
        lamports: 0,
        data: vec![],
        owner: SYSTEM_PROGRAM_ID,
        executable: false,
        rent_epoch: 0,
    }
}

```

Writing a Test

1. Build the Instruction

```
fn build_create_proposal_ix(
    program_id: &Pubkey,
    proposal: &Pubkey,
    dwallet: &Pubkey,
    creator: &Pubkey,
    payer: &Pubkey,
    proposal_id: [u8; 32],
    message_hash: [u8; 32],
    quorum: u32,
    bump: u8,
) -> Instruction {
    let mut ix_data = Vec::with_capacity(104);
    ix_data.push(0); // discriminator
    ix_data.extend_from_slice(&proposal_id);
    ix_data.extend_from_slice(&message_hash);
    ix_data.extend_from_slice(&[0u8; 32]); // user_pubkey
    ix_data.push(0); // signature_scheme
    ix_data.extend_from_slice(&quorum.to_le_bytes());
    ix_data.push(0); // message_approval_bump
    ix_data.push(bump);

    Instruction {
        program_id: *program_id,
        accounts: vec![
            AccountMeta::new(*proposal, false),
            AccountMeta::new_readonly(*dwallet, false),
            AccountMeta::new_readonly(*creator, true),
            AccountMeta::new(*payer, true),
            AccountMeta::new_readonly(SYSTEM_PROGRAM_ID, false),
        ],
        data: ix_data,
    }
}
```

2. Process and Assert

```
#[test]
fn test_create_proposal_success() {
    let (mollusk, program_id) = setup();
    let creator = Pubkey::new_unique();
    let payer = Pubkey::new_unique();
    let proposal_id = [0x01u8; 32];

    let (proposal_pda, bump) =
        Pubkey::find_program_address(&[b"proposal", &proposal_id],
&program_id);

    let ix = build_create_proposal_ix(
        &program_id, &proposal_pda, &Pubkey::new_unique(),
        &creator, &payer, proposal_id, [0x42u8; 32], 3, bump,
    );

    let result = mollusk.process_instruction(
        &ix,
        &[
            (proposal_pda, empty_account()),
            (Pubkey::new_unique(), funded_account()),
            (creator, funded_account()),
            (payer, funded_account()),
            (SYSTEM_PROGRAM_ID, system_program_account()),
        ],
    );

    assert!(result.program_result.is_ok());

    let prop_data = &result.resulting_accounts[0].1.data;
    assert_eq!(prop_data[0], 1); // discriminator
    assert_eq!(prop_data[1], 1); // version
}
```


Test Patterns

Verify Error Conditions

```
#[test]
fn test_double_vote_fails() {
    let (mollusk, program_id) = setup();
    // Pre-populate VoteRecord (voter already voted)
    let existing_vr = build_vote_record_data(&voter, &proposal_id, 1, vr_bump);

    let result = mollusk.process_instruction(
        &ix,
        &[
            (proposal_pda, program_account(&program_id, proposal_data)),
            (vote_record_pda, program_account(&program_id, existing_vr)),
            // ...
        ],
    );

    assert!(result.program_result.is_err());
}
```

Verify Field Values

```
let prop_data = &result.resulting_accounts[0].1.data;
assert_eq!(read_u32(prop_data, 163), 1, "yes_votes = 1");
assert_eq!(read_u32(prop_data, 167), 0, "no_votes = 0");
assert_eq!(prop_data[175], 0, "status = Open");
```

Running Mollusk Tests

```
cargo test -p ika-example-voting
```

Tests run in milliseconds – no validator startup required.

LiteSVM Tests

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

LiteSVM provides in-process Solana runtime testing. Unlike Mollusk, LiteSVM can:

- Deploy multiple programs
- Test CPI calls (e.g., your program calling the dWallet program)
- Process multiple transactions in sequence
- Simulate the full transaction lifecycle

LiteSVM is ideal for testing the integration between your program and the dWallet program, including the CPI authority pattern and `approve_message` calls.

Setup

```
[dev-dependencies]
litesvm = "0.4"
solana-sdk = "2"
```

Basic Test Structure

```
use litesvm::LiteSVM;
use solana_sdk::{
    instruction::{AccountMeta, Instruction},
    pubkey::Pubkey,
    signature::Keypair,
    signer::Signer,
    transaction::Transaction,
};

#[test]
fn test_voting_with_cpi() {
    let mut svm = LiteSVM::new();

    // Deploy the dWallet program
    let dwallet_program_id = Pubkey::new_unique();
    svm.deploy_program(
        dwallet_program_id,
        include_bytes!("path/to/ika_dwallet_program.so"),
    );

    // Deploy the voting program
    let voting_program_id = Pubkey::new_unique();
    svm.deploy_program(
        voting_program_id,
        include_bytes!("path/to/ika_example_voting.so"),
    );

    // Fund accounts
    let payer = Keypair::new();
    svm.airdrop(&payer.pubkey(), 10_000_000_000).unwrap();

    // ... test transactions ...
}
```

Testing the CPI Path

The key advantage of LiteSVM over Mollusk is testing the quorum -> approve_message CPI path:

```
// Step 1: Initialize dWallet program state (mock the NOA setup)
// Step 2: Create a dWallet
// Step 3: Transfer authority to voting program CPI PDA
// Step 4: Create a proposal
// Step 5: Cast votes until quorum triggers approve_message CPI

// After the final vote, verify MessageApproval was created
let ma_data = svm.get_account(&message_approval_pda).unwrap().data;
assert_eq!(ma_data[0], 14); // MessageApproval discriminator
assert_eq!(ma_data[139], 0); // status = Pending
```

When to Use LiteSVM vs Mollusk

Feature	Mollusk	LiteSVM
Speed	Fastest	Fast
CPI testing	No	Yes
Multi-program	No	Yes
Account persistence	No	Yes
Transaction sequencing	No	Yes
Error granularity	Instruction level	Transaction level

Use **Mollusk** for unit-testing individual instructions. Use **LiteSVM** when you need to test cross-program interactions or multi-step flows.

Tips

- Pre-populate accounts via `svm.set_account()` to skip setup transactions
- Use `svm.get_account()` to read account data after transactions
- Deploy both the dWallet program and your program to test the full CPI flow
- The CPI authority PDA derivation must use `b"__ika_cpi_authority"` as the seed

E2E Tests

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

E2E tests run the full dWallet lifecycle against Solana devnet and the pre-alpha dWallet gRPC service. This tests the complete flow including on-chain program execution, CPI, and signing.

Prerequisites

Resource	Endpoint
dWallet gRPC	<code>https://pre-alpha-dev-1.ika.ika-network.net:443</code>
Solana RPC	<code>https://api.devnet.solana.com</code>

Deploy your program to devnet, then run:

```
cargo run -p e2e-voting -- <DWALLET_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

Override endpoints via environment variables:

```
RPC_URL=https://api.devnet.solana.com \  
GRPC_URL=pre-alpha-dev-1.ika.ika-network.net:443 \  
cargo run -p e2e-voting -- <DWALLET_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

E2E Flow

The voting E2E demo performs 7 steps:

Step 1: Wait for Program Initialization

The mock signer creates:

- **DWalletCoordinator** PDA (["dwallet_coordinator"]) – 116 bytes
- **NetworkEncryptionKey** PDA (["network_encryption_key", noa_pubkey]) – 164 bytes

```
let (coordinator_pda, _) =
  Pubkey::find_program_address(&[b"dwallet_coordinator"],
    &dwallet_program_id);

poll_until(&client, &coordinator_pda, |data| {
  data.len() >= 116 && data[0] == 1 // DISC_COORDINATOR
}, Duration::from_secs(30));
```

Step 2: Create dWallet

The NOA commits a dWallet via `CommitDWallet` (discriminator 31):

The dWallet PDA seeds are `["dwallet", chunks_of(curve_byte || public_key)]` — concatenate the curve byte with the public key into a single buffer, then split into 32-byte chunks (Solana's `MAX_SEED_LEN`) and pass each chunk as its own seed. For 32-byte pubkeys (Curve25519/Ristretto/Ed25519) the payload is 33 bytes → chunks `[32, 1]`; for 33-byte compressed SEC1 pubkeys (Secp256k1/r1) it is 34 bytes → chunks `[32, 2]`.

```
let mut payload = Vec::with_capacity(1 + public_key.len());
payload.push(curve);
payload.extend_from_slice(&public_key);

let mut seeds: Vec<&[u8]> = Vec::with_capacity(4);
seeds.push(b"dwallet");
for chunk in payload.chunks(32) {
  seeds.push(chunk);
}

let (dwallet_pda, dwallet_bump) =
  Pubkey::find_program_address(&seeds, &dwallet_program_id);
```

Step 3: Transfer Authority

Transfer dWallet authority to the voting program's CPI PDA:

```
let (cpi_authority, _) = Pubkey::find_program_address(
    &[b"__ika_cpi_authority"],
    &voting_program_id,
);
```

Step 4: Create Proposal

Create a proposal with quorum = 3:

```
let message = b"Transfer 100 USDC to treasury";
let message_hash = keccak256(message);
```

Step 5: Cast 3 Votes

Three voters (Alice, Bob, Charlie) each cast YES. Charlie's vote reaches quorum and triggers the `approve_message` CPI.

The last vote transaction includes 10 accounts (5 base + 5 CPI accounts).

Step 6: Verify MessageApproval

```
let ma_data = client.get_account(&message_approval_pda)?.data;
assert_eq!(ma_data[0], 14); // discriminator
assert_eq!(ma_data[139], 0); // status = Pending
```

Step 7: Sign and Verify

The mock signer signs the message and calls `CommitSignature` (discriminator 43):

```
let signed_data = client.get_account(&message_approval_pda)?.data;
let sig_len = u16::from_le_bytes(signed_data[140..142].try_into().unwrap()) as
usize;
let signature = &signed_data[142..142 + sig_len];
```

Key PDA Seeds

PDA	Seeds	Program
DWalletCoordinator	["dwallet_coordinator"]	dWallet
NetworkEncryptionKey	["network_encryption_key", noa_pubkey]	dWallet
DWallet	["dwallet", chunks_of(curve public_key)] (32-byte chunks)	dWallet
MessageApproval	["message_approval", dwallet, message_hash]	dWallet
CPI Authority	["__ika_cpi_authority"]	Your program
Proposal	["proposal", proposal_id]	Voting
VoteRecord	["vote", proposal_id, voter]	Voting

Key Discriminators

Instruction	Discriminator
CreateProposal (voting)	0
CastVote (voting)	1
ApproveMessage (dWallet)	8
TransferOwnership (dWallet)	24
CommitDWallet (dWallet)	31
CommitSignature (dWallet)	43

Running the E2E Test

```
cargo run -p e2e-voting -- <DWALLET_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

Expected output:

=== dWallet Voting E2E Demo ===

[Setup] Funding payer...

> Payer: <pubkey>

[1/7] Creating dWallet via CommitDWallet...

> dWallet created: <pubkey>

[2/7] Transferring dWallet authority to voting program...

> Authority transferred to CPI PDA: <pubkey>

[3/7] Creating voting proposal (quorum=3)...

> Proposal: <pubkey>

[4/7] Vote 1/3: Alice casts YES...

[4/7] Vote 2/3: Bob casts YES...

[4/7] Vote 3/3: Charlie casts YES...

> Proposal approved (yes_votes=3)

[5/7] Verifying MessageApproval on-chain...

> MessageApproval: <pubkey>

[6/7] Signing message with NOA key and committing on-chain...

> Signature committed on-chain!

[7/7] Reading signature from MessageApproval...

> Signature: <hex>

=== E2E Test Passed! ===

Pinocchio Framework

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `ika-dwallet-pinocchio` crate provides a Pinocchio-native CPI SDK for the dWallet program. Pinocchio is the highest-performance Solana program framework — `#![no_std]`, zero-copy, minimal CU overhead.

Dependencies

```
[dependencies]
ika-dwallet-pinocchio = { git = "https://github.com/dwallet-labs/ika-pre-alpha"
}
pinocchio = "0.10"
pinocchio-system = "0.5"

[lib]
crate-type = ["cdylib", "lib"]
```

DWalletContext

```
use ika_dwallet_pinocchio::DWalletContext;

let ctx = DWalletContext {
    dwallet_program: &dwallet_program_account,
    cpi_authority: &cpi_authority_account,
    caller_program: &my_program_account,
    cpi_authority_bump: bump,
};
```

Field	Type	Description
dwallet_program	&AccountView	The dWallet program account
cpi_authority	&AccountView	Your program's CPI authority PDA
caller_program	&AccountView	Your program's account (must be executable)
cpi_authority_bump	u8	Bump seed for the CPI authority PDA

CPI Authority PDA

Every program that controls a dWallet derives a single CPI authority PDA:

```
use ika_dwallet_pinocchio::CPI_AUTHORITY_SEED;

// Derive at runtime:
let (cpi_authority, bump) = pinocchio::Address::find_program_address(
    &[CPI_AUTHORITY_SEED],
    program_id,
);
```

Methods

approve_message

Creates a `MessageApproval` PDA requesting a signature from the Ika network.

```
ctx.approve_message(
    message_approval, // &AccountView – PDA to create
    dwallet,          // &AccountView – the dWallet
    payer,            // &AccountView – pays rent
    system_program,   // &AccountView
    message_hash,     // [u8; 32]
    user_pubkey,      // [u8; 32]
    signature_scheme, // u8: 0=Ed25519, 1=Secp256k1, 2=Secp256r1
    bump,             // u8 – MessageApproval PDA bump
)?;
```

transfer_dwallet

Transfers dWallet authority to a new pubkey (or another program's CPI PDA).

```
ctx.transfer_dwallet(dwallet, &new_authority_bytes)?;
```

transfer_future_sign

Transfers the completion authority of a `PartialUserSignature`.

```
ctx.transfer_future_sign(partial_user_sig, &new_authority_bytes)?;
```

Example: Voting dWallet

Source: `chains/solana/examples/voting/pinocchio/`

```
#![no_std]
extern crate alloc;

use pinocchio::{entrypoint, AccountView, Address, ProgramResult};
use ika_dwallet_pinocchio::DWalletContext;

entrypoint!(process_instruction);
pinocchio::nstd_panic_handler!();

fn process_instruction(
    program_id: &Address,
    accounts: &[AccountView],
    data: &[u8],
) -> ProgramResult {
    match data[0] {
        0 => create_proposal(program_id, accounts, &data[1..]),
        1 => cast_vote(program_id, accounts, &data[1..]),
        _ => Err(pinocchio::error::ProgramError::InvalidInstructionData),
    }
}
```

When quorum is reached in `cast_vote`, the program constructs a `DWalletContext` and calls `approve_message` — authorizing the Ika network to sign.

When to Use Pinocchio

Consideration	Pinocchio	Native	Anchor	Quasar
CU efficiency	Best	Good	Good	Best
Binary size	Smallest	Medium	Largest	Small
no_std support	Yes	No	No	Yes

Consideration	Pinocchio	Native	Anchor	Quasar
Account validation	Manual	Manual	Declarative	Declarative
Zero-copy	Manual	No	No	Built-in
Learning curve	Steepest	Medium	Easiest	Medium

Choose Pinocchio when you need maximum CU efficiency, smallest binary size, or `no_std` compatibility. Consider [Quasar](#) if you want similar performance with declarative account validation.

Native Framework (solana-program)

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `ika-dwallet-native` crate provides a CPI SDK using Solana’s standard `solana-program` crate. No framework lock-in — just raw `AccountInfo` and `invoke_signed`.

Dependencies

```
[dependencies]
ika-dwallet-native = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
solana-program = "2.2"
solana-system-interface = "1"

[lib]
crate-type = ["cdylib", "lib"]
```

DWalletContext

```
use ika_dwallet_native::DWalletContext;

let ctx = DWalletContext {
    dwallet_program: &accounts.dwallet_program,
    cpi_authority: &accounts.cpi_authority,
    caller_program: &accounts.program,
    cpi_authority_bump: bump,
};
```

Field	Type	Description
<code>dwallet_program</code>	<code>&AccountInfo<'info></code>	The dWallet program account

Field	Type	Description
<code>cpi_authority</code>	<code>&AccountInfo<'info></code>	Your program's CPI authority PDA
<code>caller_program</code>	<code>&AccountInfo<'info></code>	Your program's account (must be executable)
<code>cpi_authority_bump</code>	<code>u8</code>	Bump seed for the CPI authority PDA

CPI Authority PDA

```
use ika_dwallet_native::CPI_AUTHORITY_SEED;
use solana_program::pubkey::Pubkey;

let (cpi_authority, bump) = Pubkey::find_program_address(
    &[CPI_AUTHORITY_SEED],
    &your_program_id,
);
```

Methods

approve_message

```
ctx.approve_message(
    &message_approval, // &AccountInfo – PDA to create
    &dwallet,           // &AccountInfo – the dWallet
    &payer,              // &AccountInfo – pays rent
    &system_program,    // &AccountInfo
    message_hash,       // [u8; 32]
    user_pubkey,        // [u8; 32]
    signature_scheme,   // u8
    bump,               // u8 – MessageApproval PDA bump
)?;
```

transfer_dwallet

```
ctx.transfer_dwallet(&dwallet, &new_authority)?;
```

transfer_future_sign

```
ctx.transfer_future_sign(&partial_user_sig, &new_authority)?;
```

Example: Voting dWallet

Source: `chains/solana/examples/voting/native/`

```
use solana_program::{
    account_info::{next_account_info, AccountInfo},
    entrypoint,
    entrypoint::ProgramResult,
    program::invoke_signed,
    program_error::ProgramError,
    pubkey::Pubkey,
    rent::Rent,
    sysvar::Sysvar,
};
use ika_dwallet_native::DWalletContext;

entrypoint!(process_instruction);

fn process_instruction(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    instruction_data: &[u8],
) -> ProgramResult {
    match instruction_data[0] {
        0 => create_proposal(program_id, accounts, &instruction_data[1..]),
        1 => cast_vote(program_id, accounts, &instruction_data[1..]),
        _ => Err(ProgramError::InvalidInstructionData),
    }
}
```

Uses `next_account_info()` for account iteration, `Rent::get()?.minimum_balance()` for rent, and `system_instruction::create_account + invoke_signed` for PDA creation.

When quorum is reached, the program constructs a `DWalletContext` and calls `approve_message`.

When to Use Native

Consideration	Pinocchio	Native	Anchor	Quasar
CU efficiency	Best	Good	Good	Best

Consideration	Pinocchio	Native	Anchor	Quasar
std library	No (no_std)	Yes	Yes	No (no_std)
Framework dependency	pinocchio	solana-program	anchor-lang	quasar-lang
Account validation	Manual	Manual	Declarative	Declarative
Migration from existing	Rewrite	Minimal	Rewrite	Rewrite

Choose Native when you have an existing `solana-program` codebase, want `std` library access, or prefer no framework lock-in beyond Solana's standard SDK.

Differences from Pinocchio

	Pinocchio	Native
Account type	<code>&AccountView</code>	<code>&AccountInfo<'info></code>
Entrypoint	<code>pinocchio::entrypoint!()</code>	<code>solana_program::entrypoint!</code>
CPI	<code>pinocchio::cpi::invoke_signed</code>	<code>solana_program::program::in</code>
PDA creation	<code>pinocchio_system::CreateAccount</code>	<code>system_instruction::create_ invoke_signed</code>
Rent	<code>minimum_balance()</code> helper	<code>Rent::get()?.minimum_balanc</code>
std	<code>#![no_std]</code>	Full std
Account iteration	Array indexing	<code>next_account_info()</code>

All four SDKs (Pinocchio, Native, Anchor, [Quasar](#)) use the same CPI authority seed, instruction discriminators, and account layouts. Programs built with any SDK are fully interoperable.

Anchor Framework (v1.0.0)

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real MPC signing – all signatures are generated by a single mock signer, not a distributed network. Do not submit any real transactions for signing or rely on any security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `ika-dwallet-anchor` crate provides an Anchor-native CPI SDK for the dWallet program. It is the Anchor equivalent of `ika-dwallet-pinocchio`.

Anchor v1.0.0: This SDK uses Anchor’s first stable release ([release notes](#)). Key v1 features used:

- `UncheckedAccount` instead of raw `AccountInfo` in `#[derive(Accounts)]`
- `InitSpace` derive for automatic space calculation
- Single `#[error_code]` block per program
- Solana 3.x compatibility

Dependencies

`[dependencies]`

```
ika-dwallet-anchor = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }  
anchor-lang = "1"
```

`[lib]`

```
crate-type = ["cdylib", "lib"]
```

Note: Anchor v1 requires Solana CLI 3.x and the Anchor CLI 1.x. Install with:

```
cargo install --git https://github.com/coral-xyz/anchor avm --force  
avm install 1.0.0  
avm use 1.0.0
```

DWalletContext

The `DWalletContext` struct wraps the accounts needed for CPI calls to the dWallet program.

```
use ika_dwallet_anchor::{DWalletContext, CPI_AUTHORITY_SEED};

let ctx = DWalletContext {
    dwallet_program: dwallet_program.to_account_info(),
    cpi_authority: cpi_authority.to_account_info(),
    caller_program: program.to_account_info(),
    cpi_authority_bump: bump,
};
```

Field	Type	Description
<code>dwallet_program</code>	<code>AccountInfo</code>	The dWallet program account
<code>cpi_authority</code>	<code>AccountInfo</code>	Your program's CPI authority PDA
<code>caller_program</code>	<code>AccountInfo</code>	Your program's account (must be executable)
<code>cpi_authority_bump</code>	<code>u8</code>	Bump seed for the CPI authority PDA

CPI Authority PDA

Same derivation as Pinocchio – a single seed per program:

```
use ika_dwallet_anchor::CPI_AUTHORITY_SEED;

let (cpi_authority, bump) = Pubkey::find_program_address(
    &[CPI_AUTHORITY_SEED],
    &your_program_id,
);
```

Methods

approve_message

Creates a `MessageApproval` PDA requesting a signature.

```
ctx.approve_message(
    &message_approval.to_account_info(),
    &dwallet.to_account_info(),
    &payer.to_account_info(),
    &system_program.to_account_info(),
    message_hash,          // [u8; 32]
    user_pubkey,           // [u8; 32]
    signature_scheme,      // u8: 0=Ed25519, 1=Secp256k1, 2=Secp256r1
    bump,                  // MessageApproval PDA bump
)?;
```

transfer_dwallet

Transfers dWallet authority to a new pubkey.

```
ctx.transfer_dwallet(
    &dwallet.to_account_info(),
    &new_authority,        // &Pubkey
)?;
```

transfer_future_sign

Transfers the completion authority of a `PartialUserSignature`.

```
ctx.transfer_future_sign(
    &partial_user_sig.to_account_info(),
    &new_authority,        // &Pubkey
)?;
```

Example: Voting-Controlled dWallet

The `voting-anchor` example demonstrates the full pattern. Proposals reference a dWallet whose authority has been transferred to this program's CPI authority PDA. When enough yes-votes reach quorum, the program CPI-calls `approve_message`.

Source: `chains/solana/examples/voting-anchor/`

Account Definitions (Anchor v1 style)

```
use anchor_lang::prelude::*;

#[account]
#[derive(InitSpace)]    // v1: auto-calculates space
pub struct Proposal {
    pub proposal_id: [u8; 32],
    pub dwallet: Pubkey,
    pub message_hash: [u8; 32],
    pub user_pubkey: [u8; 32],
    pub signature_scheme: u8,
    pub creator: Pubkey,
    pub yes_votes: u32,
    pub no_votes: u32,
    pub quorum: u32,
    pub status: ProposalStatus,
    pub message_approval_bump: u8,
}

#[account]
#[derive(InitSpace)]
pub struct VoteRecord {
    pub voter: Pubkey,
    pub proposal_id: [u8; 32],
    pub vote: bool,
}
```

Account Validation (Anchor v1 constraints)

```
#[derive(Accounts)]
#[instruction(proposal_id: [u8; 32])]
pub struct CreateProposal<'info> {
    #[account(
        init,
        payer = payer,
        space = 8 + Proposal::INIT_SPACE,    // v1: InitSpace derive
        seeds = [b"proposal", proposal_id.as_ref()],
        bump,
    )]
    pub proposal: Account<'info, Proposal>,
    /// CHECK: dWallet account (owned by dWallet program)
    pub dwallet: UncheckedAccount<'info>,    // v1: UncheckedAccount
    pub creator: Signer<'info>,
    #[account(mut)]
    pub payer: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

CPI on Quorum (cast_vote)

```
pub fn cast_vote(
    ctx: Context<CastVote>,
    proposal_id: [u8; 32],
    vote: bool,
    cpi_authority_bump: u8,
) -> Result<()> {
    let proposal = &mut ctx.accounts.proposal;
    require!(proposal.status == ProposalStatus::Open,
VotingError::ProposalClosed);

    if vote {
        proposal.yes_votes = proposal.yes_votes.checked_add(1)
            .ok_or(VotingError::ProposalClosed)?;
    } else {
        proposal.no_votes = proposal.no_votes.checked_add(1)
            .ok_or(VotingError::ProposalClosed)?;
    }

    // Quorum reached → CPI approve_message
    if proposal.yes_votes >= proposal.quorum {
        let dwallet_ctx = DWalletContext {
            dwallet_program: ctx.accounts.dwallet_program.to_account_info(),
            cpi_authority: ctx.accounts.cpi_authority.to_account_info(),
            caller_program: ctx.accounts.program.to_account_info(),
            cpi_authority_bump,
        };

        dwallet_ctx.approve_message(
            &ctx.accounts.message_approval.to_account_info(),
            &ctx.accounts.dwallet.to_account_info(),
            &ctx.accounts.payer.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            proposal.message_hash,
            proposal.user_pubkey,
            proposal.signature_scheme,
            proposal.message_approval_bump,
        )?;

        proposal.status = ProposalStatus::Approved;
    }

    Ok(())
}
```

Error Definition (v1: single block only)

```
#[error_code]
pub enum VotingError {
    #[msg("Proposal is not open for voting")]
    ProposalClosed,
}
```

Anchor v1 enforces a single `#[error_code]` block per program. Multiple blocks now produce a compile-time error.

Key Patterns

PDA-based proposals — each proposal is a PDA seeded by `[b"proposal", proposal_id]`.

One vote per voter — vote records are PDAs seeded by `[b"vote", proposal_id, voter_pubkey]`, preventing double-voting via Anchor's `init` constraint.

Automatic CPI on quorum — when `yes_votes >= quorum`, `cast_vote` constructs a `DWalletContext` and calls `approve_message` in the same transaction.

UncheckedAccount for cross-program accounts — `dWallet`-program-owned accounts use `UncheckedAccount` with `/// CHECK: comments` (v1 best practice, replacing raw `AccountInfo`).

Anchor v1.0.0 Migration Notes

If migrating from Anchor 0.30/0.31:

Change	Before (0.30)	After (v1)
Space calculation	Manual <code>8 + 32 + 32 + ...</code>	<code>8 + MyAccount::INIT_SPACE</code> (InitSpace derived)
Raw AccountInfo	<code>AccountInfo<'info></code> in derives	<code>UncheckedAccount</code> with <code>/// CHECK: comments</code>
Error blocks	Multiple <code>#[error_code]</code> allowed	Single <code>#[error_code]</code> per program
CPI program	<code>CpiContext::new(program.to_account_info(), ...)</code>	<code>CpiContext::new(program, ...)</code> or direct
Solana version	Solana 2.x	Solana 3.x

Differences from Pinocchio SDK

Pinocchio		Anchor v1
Account types	<code>&AccountView</code>	<code>AccountInfo</code> / <code>UncheckedAccount</code>

	Pinocchio	Anchor v
Error handling	ProgramResult	anchor_lang::Result<()>
CPI signing	pinocchio::cpi::invoke_signed	anchor_lang::solana_program::
Entrypoint	Manual entrypoint!() macro	#[program] attribute macro
Account validation	Manual checks	#[derive(Accounts)] constraint:
Space	core::mem::size_of::<T>()	8 + T::INIT_SPACE (InitSpace
Best for	Maximum CU efficiency	Rapid development, safety

All four SDKs (Pinocchio, Native, Anchor, [Quasar](#)) use the same CPI authority seed (`b"__ika_cpi_authority"`), the same instruction discriminators, and the same account layouts. Programs built with any SDK are fully interoperable.

Quasar Framework

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real MPC signing – all signatures are generated by a single mock signer, not a distributed network. Do not submit any real transactions for signing or rely on any security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `ika-dwallet-quasar` crate provides a Quasar-native CPI SDK for the dWallet program. [Quasar](#) is a zero-copy Solana program framework with alignment-1 Pod types, declarative account validation, and low-level CPI control via `invoke_signed_unchecked`.

Dependencies

```
[dependencies]
ika-dwallet-quasar = { git = "https://github.com/dwallet-labs/ika-pre-alpha" }
quasar-lang = { git = "https://github.com/blueshift-gg/quasar", branch = "master" }
solana-address = { version = "2.4", features = ["curve25519"] }
```

```
[lib]
crate-type = ["cdylib", "lib"]
```

DWalletContext

```
use ika_dwallet_quasar::DWalletContext;

let ctx = DWalletContext {
    dwallet_program: self.dwallet_program.to_account_view(),
    cpi_authority: self.cpi_authority.to_account_view(),
    caller_program: self.caller_program.to_account_view(),
    cpi_authority_bump: bump,
};
```

Field	Type	Description
dwallet_program	&AccountView	The dWallet program account
cpi_authority	&AccountView	Your program's CPI authority PDA
caller_program	&AccountView	Your program's account (must be executable)
cpi_authority_bump	u8	Bump seed for the CPI authority PDA

Convert Quasar account types to `&AccountView` using `.to_account_view()` (available on `Signer`, `UncheckedAccount`, `Program<T>`, `Account<T>`).

CPI Authority PDA

```
use ika_dwallet_quasar::CPI_AUTHORITY_SEED;
use solana_address::Address;

let (cpi_authority, bump) = Address::find_program_address(
    &[CPI_AUTHORITY_SEED],
    &your_program_id,
);
```

Methods

approve_message

Creates a `MessageApproval` PDA requesting a signature.

```
ctx.approve_message(
    self.coordinator.to_account_view(),
    self.message_approval.to_account_view(),
    self.dwallet.to_account_view(),
    self.payer.to_account_view(),
    self.system_program.to_account_view(),
    message_digest,           // [u8; 32]
    message_metadata_digest, // [u8; 32] -- zero if no metadata
    user_pubkey,             // [u8; 32]
    signature_scheme,        // u16
    bump,                    // u8 -- MessageApproval PDA bump
)?;
```

transfer_dwallet

Transfers dWallet authority to a new pubkey.

```
ctx.transfer_dwallet(  
    self.dwallet.to_account_view(),  
    new_authority, // [u8; 32]  
)?;
```

transfer_future_sign

Transfers the completion authority of a `PartialUserSignature`.

```
ctx.transfer_future_sign(  
    self.partial_user_sig.to_account_view(),  
    new_authority, // [u8; 32]  
)?;
```

Example: Voting-Controlled dWallet

Source: `chains/solana/examples/voting/quasar/`

Quasar programs use `#[program]` with explicit instruction discriminators, owned account types (no lifetimes), and `impl` handlers on account structs:

```

#[no_std]

use ika_dwallet_quasar::DWalletContext;
use quasar_lang::prelude::*;
use solana_address::Address;

declare_id!("US517G5965aydkZ46HS38QLi7UQiSojurfQfKCELFx");

#[program]
mod voting_quasar {
    use super::*;

    #[instruction(discriminator = 0)]
    pub fn create_proposal(
        ctx: Ctx<CreateProposal>,
        message_digest: [u8; 32],
        /* ... */
    ) -> Result<(), ProgramError> {
        ctx.accounts.create(message_digest, /* ... */)
    }

    #[instruction(discriminator = 1)]
    pub fn cast_vote(
        ctx: Ctx<CastVote>,
        vote: bool,
        cpi_authority_bump: u8,
    ) -> Result<(), ProgramError> {
        ctx.accounts.cast(vote, cpi_authority_bump)
    }
}

```

Account Definitions (Quasar style)

Quasar uses owned types (no lifetime parameters), zero-copy `#[account]` with explicit discriminators, and `#[seeds]` on state structs:

```

#[account(discriminator = 1, set_inner)]
#[seeds(b"proposal", proposal_id: Address)]
pub struct Proposal {
    pub proposal_id: Address,
    pub dwallet: Address,
    pub message_digest: [u8; 32],
    pub user_pubkey: [u8; 32],
    pub signature_scheme: u16,
    pub creator: Address,
    pub yes_votes: u32,
    pub no_votes: u32,
    pub quorum: u32,
    pub status: u8,
    pub message_approval_bump: u8,
}

```

Account Validation (Quasar constraints)

```
#[derive(Accounts)]
pub struct CastVote {
    pub proposal_id: UncheckedAccount,

    #[account(mut, seeds = Proposal::seeds(proposal_id), bump)]
    pub proposal: Account<Proposal>,

    #[account(init, payer = payer, seeds = VoteRecord::seeds(proposal_id,
voter), bump)]
    pub vote_record: Account<VoteRecord>,

    pub voter: Signer,

    #[account(mut)]
    pub payer: Signer,

    pub system_program: Program<System>,

    // CPI accounts for dWallet interaction
    pub coordinator: UncheckedAccount,
    #[account(mut)]
    pub message_approval: UncheckedAccount,
    pub dwallet: UncheckedAccount,
    pub caller_program: UncheckedAccount,
    pub cpi_authority: UncheckedAccount,
    pub dwallet_program: UncheckedAccount,
}
```

Handler Pattern (impl on Accounts struct)

```
impl CastVote {
    pub fn cast(&mut self, vote: bool, cpi_authority_bump: u8) -> Result<(),
ProgramError> {
        // Mutate fields via zero-copy Pod types
        self.proposal.yes_votes = self.proposal.yes_votes
            .checked_add(1u32)
            .ok_or(VotingError::ArithmeticOverflow)?;

        // CPI when quorum reached
        if self.proposal.yes_votes >= self.proposal.quorum {
            let dwallet_ctx = DWalletContext {
                dwallet_program: self.dwallet_program.to_account_view(),
                cpi_authority: self.cpi_authority.to_account_view(),
                caller_program: self.caller_program.to_account_view(),
                cpi_authority_bump,
            };
            dwallet_ctx.approve_message(/* ... */)?;
            self.proposal.status = 1; // Approved
        }
        Ok(())
    }
}
```

Error Codes

```
#[error_code]
pub enum VotingError {
    ProposalClosed = 6000,
    InvalidQuorum,
    ArithmeticOverflow,
}
```

Key Patterns

Seed components as accounts – Quasar resolves PDA seeds from account addresses. Pass seed values (like `proposal_id`) as `UncheckedAccount` fields whose addresses are the seed bytes.

Owned account types – `Signer`, `UncheckedAccount`, `Program<System>`, `Account<T>` are owned (no lifetime parameters or references).

Pod arithmetic – Multi-byte fields are zero-copy Pod types (`PodU16`, `PodU32`). Use `.checked_add()`, `.into()`, and direct comparison operators.

set_inner() for initialization – The `#[account(set_inner)]` macro generates a companion `Inner` struct with original Rust types for initialization.

Stack-allocated CPI – The Quasar SDK uses `invoke_signed_unchecked` with stack-allocated buffers (no heap allocation), making it the most CU-efficient CPI variant.

When to Use Quasar

Consideration	Pinocchio	Native	Anchor	Quasar
CU efficiency	Best	Good	Good	Best
Binary size	Smallest	Medium	Largest	Small
no_std support	Yes	No	No	Yes
Account validation	Manual	Manual	Declarative	Declarative
Zero-copy	Manual	No	No	Built-in
Learning curve	Steepest	Medium	Easiest	Medium

Choose Quasar when you want declarative account validation (like Anchor) combined with zero-copy performance (like Pinocchio), built-in Pod types for safe zero-copy field access, and `no_std` compatibility.

Differences from Other SDKs

	Pinocchio	Anchor	Quasar
Account types	<code>&AccountView</code>	<code>AccountInfo</code>	<code>Owned Signer / UncheckedAccount</code>
CPI	<code>invoke_signed</code>	<code>invoke_signed</code> (via <code>solana_program</code>)	<code>invoke_signed_unchecked</code>
CPI data	Heap Vec	Heap Vec	Stack <code>[u8; N]</code>
Instruction dispatch	Manual match	<code>#[program]</code> (hash-based)	<code>#[program] + # [instruction(discriminator = N)]</code>
Field access	Raw byte offsets	Borsh deserialized	Zero-copy Pod types
Account init	Manual <code>CreateAccount</code>	<code># [account(init)]</code>	<code>#[account(init)]</code>
Error handling	<code>ProgramResult</code>	<code>Result<()></code>	<code>Result<(), ProgramError></code>

All four SDKs use the same CPI authority seed (`b"__ika_cpi_authority"`), the same instruction discriminators, and the same wire format. Programs built with any SDK are fully

interoperable at the dWallet program level.

TypeScript Client

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `@ika.xyz/pre-alpha-solana-client` package provides a TypeScript client for interacting with the dWallet program on Solana. Built on `@solana/kit` (web3.js v2).

Installation

```
bun add @ika.xyz/pre-alpha-solana-client @solana/kit
```

Or with npm:

```
npm install @ika.xyz/pre-alpha-solana-client @solana/kit
```

Quick Start

```
import {
  address,
  createSolanaRpc,
  createSolanaRpcSubscriptions,
  generateKeyPairSigner,
  pipe,
  createTransactionMessage,
  setTransactionMessageFeePayerSigner,
  setTransactionMessageLifetimeUsingBlockhash,
  appendTransactionMessageInstruction,
  signTransactionMessageWithSigners,
  sendAndConfirmTransactionFactory,
} from "@solana/kits";

const RPC_URL = "https://api.devnet.solana.com";
const WS_URL = "wss://api.devnet.solana.com";
const DWALLET_PROGRAM =
  address("87W54kGYFQ1rgWqMeu4XTPHWXWmXSQCcjm8vCTfiq1oY");

const rpc = createSolanaRpc(RPC_URL);
const rpcSubscriptions = createSolanaRpcSubscriptions(WS_URL);
const sendAndConfirm = sendAndConfirmTransactionFactory({ rpc, rpcSubscriptions
});
```

Building Transactions

Approve Message

Build an `ApproveMessage` instruction to authorize the Ika network to sign a message:

```

import { getAddressEncoder, getProgramDerivedAddress, getUtf8Encoder } from
"@solana/kits";

const utf8 = getUtf8Encoder();
const addressEncoder = getAddressEncoder();

// Derive MessageApproval PDA
const [messageApprovalPda, messageApprovalBump] = await
getProgramDerivedAddress({
  seeds: [
    utf8.encode("message_approval"),
    addressEncoder.encode(dwalletAddress),
    messageHash, // Uint8Array(32)
  ],
  programAddress: DWALLET_PROGRAM,
});

// Build instruction data: disc(1) + bump(1) + message_hash(32) +
user_pubkey(32) + scheme(1) = 67
const data = new Uint8Array(67);
data[0] = 8; // IX_APPROVE_MESSAGE discriminator
data[1] = messageApprovalBump;
data.set(messageHash, 2);
data.set(userPubkey, 34);
data[66] = 0; // signature_scheme: 0=Ed25519

const approveMessageIx = {
  programAddress: DWALLET_PROGRAM,
  accounts: [
    { address: messageApprovalPda, role: AccountRole.WRITABLE },
    { address: dwalletAddress, role: AccountRole.READONLY },
    { address: authority, role: AccountRole.READONLY_SIGNER },
    { address: payer, role: AccountRole.WRITABLE_SIGNER },
    { address: SYSTEM_PROGRAM, role: AccountRole.READONLY },
  ],
  data,
};

```

Transfer dWallet Authority

```

const data = new Uint8Array(33);
data[0] = 24; // IX_TRANSFER_DWALLET discriminator
data.set(newAuthorityBytes, 1);

const transferIx = {
  programAddress: DWALLET_PROGRAM,
  accounts: [
    { address: currentAuthority, role: AccountRole.READONLY_SIGNER },
    { address: dwalletAddress, role: AccountRole.WRITABLE },
  ],
  data,
};

```

Send Transaction

```
const { value: latestBlockhash } = await rpc.getLatestBlockhash().send();

const tx = pipe(
  createTransactionMessage({ version: 0 }),
  (msg) => setTransactionMessageFeePayerSigner(payer, msg),
  (msg) => setTransactionMessageLifetimeUsingBlockhash(latestBlockhash, msg),
  (msg) => appendTransactionMessageInstruction(approveMessageIx, msg),
);

const signedTx = await signTransactionMessageWithSigners(tx);
await sendAndConfirm(signedTx, { commitment: "confirmed" });
```

Reading Accounts

Read dWallet

```
const account = await rpc.getAccountInfo(dwalletAddress, { encoding: "base64"
}).send();
const data = Buffer.from(account.value.data[0], "base64");

// Field offsets (after 2-byte disc+version prefix):
const authority = data.subarray(2, 34);           // [u8; 32]
const curve = data[34];                          // u8
const state = data[35];                          // u8: 0=DKGInProgress,
1=Active, 2=Frozen
const publicKeyLen = data[36];                    // u8
const publicKey = data.subarray(37, 37 + publicKeyLen);
```

Read MessageApproval

```
const data = Buffer.from(account.value.data[0], "base64");

const dwallet = data.subarray(2, 34);
const messageHash = data.subarray(34, 66);
const approver = data.subarray(66, 98);
const status = data[139];                        // 0=Pending, 1=Signed
const signatureLen = data.readUInt16LE(140);
const signature = data.subarray(142, 142 + signatureLen);
```

gRPC Client

For submitting dWallet operations (DKG, Sign, Presign) via gRPC:

```
// The gRPC client uses BCS-serialized request/response types.
// See the gRPC API section for details on SubmitTransaction.

// Connect to the pre-alpha dWallet gRPC service
const GRPC_URL = "pre-alpha-dev-1.ika.ika-network.net:443";

// gRPC types are defined in proto/ika_dwallet.proto
// Use a gRPC client library (e.g., @grpc/grpc-js or connectrpc) to call:
//   DWalletService.SubmitTransaction(UserSignedRequest) -> TransactionResponse
```

Instruction Discriminators

Discriminator	Instruction
8	ApproveMessage
24	TransferDWallet
31	CommitDWallet
33	CommitFutureSign
34	CommitEncryptedUserSecretKeyShare
35	CommitPublicUserSecretKeyShare
36	CreateDeposit
37	TopUp
38	SettleGas
42	TransferFutureSign
43	CommitSignature
44	RequestWithdraw
45	Withdraw
46	Initialize

PDA Seeds

Account	Seeds
DWalletCoordinator	["dwallet_coordinator"]
DWallet	["dwallet", chunks_of(curve_byte public_key)] (32-byte chunks)

Account	Seeds
MessageApproval	["message_approval", dwallet_pubkey, message_hash]
GasDeposit	["gas_deposit", user_pubkey]
NetworkEncryptionKey	["network_encryption_key", noa_public_key]
CPI Authority	["__ika_cpi_authority"] (derived per calling program)

Framework Comparison

	Pinocchio	Native	Anchor	T
Language	Rust (no_std)	Rust (std)	Rust (std)	T
Runs	On-chain	On-chain	On-chain	C
Use case	Program CPI	Program CPI	Program CPI	C tr
Account types	AccountView	AccountInfo	Account / UncheckedAccount	A rā
Best for	Max performance	Existing codebases	Rapid development	d S

Examples

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

The Ika SDK ships with two complete example programs demonstrating different patterns for program-controlled dWallet signing:

Example	Pattern	Description
Voting	Governance	Open voting with quorum — anyone can vote, quorum triggers signing
Multisig	Access Control	M-of-N multisig — fixed members, threshold approval triggers signing

Structure

Each example follows the same directory layout:

```

examples/<name>/
├── pinocchio/           # Pinocchio framework implementation
│   ├── src/lib.rs
│   └── tests/mollusk.rs
├── native/             # Native solana-program implementation
│   ├── src/lib.rs
│   └── tests/mollusk.rs
├── anchor/             # Anchor framework implementation
│   └── src/lib.rs
├── quasar/             # Quasar framework implementation
│   └── src/lib.rs
├── e2e/                # TypeScript end-to-end tests (bun)
│   ├── main.ts
│   └── instructions.ts
├── e2e-rust/           # Rust end-to-end tests (alternative)
│   └── src/main.rs

```

Common Flow

Both examples follow the same high-level flow:

1. **Create dWallet** via gRPC DKG request — the mock commits on-chain and transfers authority to the caller
2. **Transfer authority** to the example program's CPI authority PDA
(["__ika_cpi_authority"])
3. **Create proposal/transaction** — on-chain state describing what to sign
4. **Collect approvals** — votes or multisig member approvals
5. **CPI approve_message** — when threshold is reached, the program calls the dWallet program to create a `MessageApproval` PDA
6. **gRPC presign + sign** — allocate a presign, then sign via gRPC with `ApprovalProof` referencing the on-chain approval
7. **Signature returned** — 64-byte Ed25519 signature for the approved message

Running Examples

Pre-Alpha Environment

Resource	Endpoint
dWallet gRPC	https://pre-alpha-dev-1.ika.ika-network.net:443
Solana RPC	https://api.devnet.solana.com

Deploy your example program to devnet, then run E2E tests against the pre-alpha environment:

```
# TypeScript (recommended)
just e2e-voting <DWALLET_ID> <VOTING_ID>
just e2e-multisig <DWALLET_ID> <MULTISIG_ID>

# Rust (alternative)
just e2e-voting-rust <DWALLET_ID> <VOTING_ID>
just e2e-multisig-rust <DWALLET_ID> <MULTISIG_ID>
```

Unit Tests (Mollusk)

Mollusk tests run in-process with no network dependency:

```
just test-examples-mollusk
```

Shared Helpers

TypeScript e2e tests use shared helpers from `examples/_shared/`:

- **helpers.ts** — Colored logging (`log`, `ok`, `val`), `sendTx`, `pda`, `pollUntil`, `createAndFundKeypair`
- **ika-setup.ts** — BCS types matching `ika-dwallet-types`, gRPC client, `setupDWallet()`, `requestPresign()`, `requestSign()`

These helpers handle the full dWallet lifecycle (DKG, on-chain polling, authority transfer) so example e2e tests focus on their specific program logic.

Voting Example

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

A governance program where anyone can create proposals referencing a dWallet. Voters cast yes/no votes, and when the quorum of yes votes is reached, the program automatically CPI-calls `approve_message` on the dWallet program to authorize signing.

Use case: DAO governance, treasury management, group signing authorization.

Program Design

Accounts

Account	Seeds	Size	Description
Proposal	["proposal", proposal_id]	195 bytes	Stores vote counts, quorum, message hash, dWallet reference
VoteRecord	["vote", proposal_id, voter]	69 bytes	Prevents double voting — one per voter per proposal

Instructions

Disc	Instruction	Description
0	CreateProposal	Create a proposal with message hash, quorum, and dWallet reference
1	CastVote	Record a vote; when quorum reached, CPI-calls <code>approve_message</code>

Proposal Layout (195 bytes)

`disc(1) + version(1) + proposal_id(32) + dwallet(32) + message_hash(32) + user_pubkey(32) + signature_scheme(1) + creator(32) + yes_votes(4) + no_votes(4) + quorum(4) + status(1) + message_approval_bump(1) + bump(1) + _reserved(16)`

Key Offsets

Field	Offset	Size	Type
yes_votes	163	4	u32 LE
no_votes	167	4	u32 LE
quorum	171	4	u32 LE
status	175	1	0=Open, 1=Approved

CPI Flow

When `yes_votes >= quorum`, the program:

1. Reads message hash, user pubkey, and signature scheme from the Proposal account
2. Constructs a `DWalletContext` with the CPI authority PDA
3. Calls `ctx.approve_message(...)` which creates a `MessageApproval` PDA on the dWallet program
4. Sets proposal status to Approved

The caller then sends a gRPC `Sign` request with `ApprovalProof::Solana { transaction_signature, slot }` to obtain the actual signature.

E2E Flow

1. gRPC DKG → dWallet created on-chain, authority = caller
2. Transfer authority → CPI PDA owns the dWallet
3. Create proposal → quorum=3, message="Transfer 100 USDC"
4. Cast 3 YES votes → last vote triggers approve_message CPI
5. Verify approval → MessageApproval PDA exists, status=Pending
6. gRPC presign → allocate presign
7. gRPC sign → 64-byte signature returned

Testing

```
# Mollusk (instruction-level, no infrastructure needed)
cargo test -p ika-example-voting-pinocchio --test mollusk
cargo test -p ika-example-voting-native --test mollusk

# TypeScript E2E (requires validator + mock)
cd chains/solana/examples/voting/e2e && bun main.ts <DWALLET_ID> <VOTING_ID>
```

Source Files

- Pinocchio: chains/solana/examples/voting/pinocchio/src/lib.rs
- Native: chains/solana/examples/voting/native/src/lib.rs
- Anchor: chains/solana/examples/voting/anchor/src/lib.rs
- TypeScript E2E: chains/solana/examples/voting/e2e/main.ts

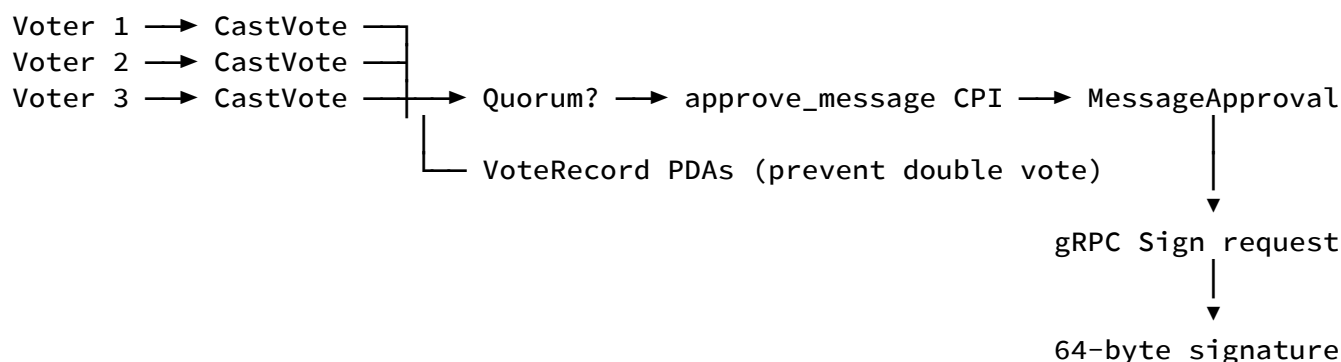
Building the Voting Program

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What You’ll Learn

- How to define on-chain account layouts for proposals and vote records
- How to implement the CPI authority pattern for dWallet control
- How quorum detection triggers `approve_message` via CPI
- How to prevent double voting using PDA-based vote records

Architecture



1. Account Layouts

Proposal PDA(["proposal", proposal_id]) — 195 bytes

```
// Header
discriminator: u8,      // offset 0, always 1
version: u8,            // offset 1, always 1

// Fields
proposal_id: [u8; 32], // offset 2
dwallet: [u8; 32],     // offset 34 – the dWallet this proposal controls
message_hash: [u8; 32], // offset 66 – keccak256 of the message to sign
user_pubkey: [u8; 32], // offset 98
signature_scheme: u8,  // offset 130
creator: [u8; 32],     // offset 131
yes_votes: u32,        // offset 163 (LE)
no_votes: u32,         // offset 167 (LE)
quorum: u32,           // offset 171 (LE)
status: u8,            // offset 175 – 0=Open, 1=Approved
msg_approval_bump: u8, // offset 176
bump: u8,              // offset 177
_reserved: [u8; 16],   // offset 178
```

VoteRecord PDA(["vote", proposal_id, voter]) — 69 bytes

```
discriminator: u8,      // offset 0, always 2
version: u8,            // offset 1

voter: [u8; 32],        // offset 2
proposal_id: [u8; 32],  // offset 34
vote: u8,               // offset 66 – 1=yes, 0=no
bump: u8,               // offset 67
```

2. CreateProposal Instruction (disc = 0)

Data: [proposal_id(32), message_hash(32), user_pubkey(32), signature_scheme(1), quorum(4), message_approval_bump(1), bump(1)] = 103 bytes

Accounts:

#	Account	Flags	Description
0	Proposal PDA	writable	Created via <code>invoke_signed</code>
1	dWallet	readonly	The dWallet account on the dWallet program

#	Account	Flags	Description
2	Creator	signer	Proposal authority
3	Payer	writable, signer	Pays rent
4	System Program	readonly	For PDA creation

3. CastVote Instruction (disc = 1)

Data: [proposal_id(32), vote(1), vote_record_bump(1), cpi_authority_bump(1)] = 35 bytes

Base accounts (always required):

#	Account	Flags
0	Proposal PDA	writable
1	VoteRecord PDA	writable
2	Voter	signer
3	Payer	writable, signer
4	System Program	readonly

CPI accounts (when quorum will be reached):

#	Account	Flags
5	MessageApproval PDA	writable
6	dWallet	readonly
7	Voting Program	readonly
8	CPI Authority PDA	readonly
9	dWallet Program	readonly

4. The CPI Call

When `yes_votes >= quorum`, the program:

```
let ctx = DWalletContext {  
    dwallet_program,  
    cpi_authority,  
    caller_program,  
    cpi_authority_bump,  
};  
  
ctx.approve_message(  
    message_approval, dwallet, payer, system_program,  
    message_hash, user_pubkey, signature_scheme,  
    message_approval_bump,  
)?;  
  
prop_data[PROP_STATUS] = STATUS_APPROVED;
```

The `DWalletContext` signs via `invoke_signed` with seeds `["__ika_cpi_authority", &[bump]]`, proving the voting program authorized this signing request.

Source Code

Framework	Path
Pinocchio	<code>chains/solana/examples/voting/pinocchio/src/lib.rs</code>
Native	<code>chains/solana/examples/voting/native/src/lib.rs</code>
Anchor	<code>chains/solana/examples/voting/anchor/src/lib.rs</code>

Testing the Voting Program

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What You’ll Learn

- How to write Mollusk instruction-level tests for dWallet programs
- How to build and verify account data at specific byte offsets
- Testing error conditions (double vote, closed proposals)

Test Matrix

Test	Instruction	Expected Result
test_create_proposal_success	CreateProposal	PDA created with correct fields
test_create_proposal_already_exists	CreateProposal	Fails (account in use)
test_cast_vote_yes_success	CastVote (yes)	yes_votes incremented, status=Open
test_cast_vote_no_success	CastVote (no)	no_votes incremented, status=Open
test_cast_vote_double_vote_fails	CastVote	Fails (VoteRecord exists)
test_cast_vote_closed_proposal_fails	CastVote	Fails (status=Approved)

Running Tests

```
# Pinocchio (requires SBF build first)
cargo build-sbf --manifest-path
chains/solana/examples/voting/pinocchio/Cargo.toml
cargo test -p ika-example-voting-pinocchio --test mollusk

# Native
cargo build-sbf --manifest-path chains/solana/examples/voting/native/Cargo.toml
cargo test -p ika-example-voting-native --test mollusk
```

Key Patterns

Building Test Account Data

Tests pre-populate account data with exact byte layouts:

```
fn build_proposal_data(
    proposal_id: &[u8; 32], dwallet: &Pubkey,
    message_hash: &[u8; 32], authority: &Pubkey,
    yes_votes: u32, no_votes: u32, quorum: u32,
    status: u8, bump: u8,
) -> Vec<u8> {
    let mut data = vec![0u8; PROPOSAL_LEN]; // 195 bytes
    data[0] = PROPOSAL_DISCRIMINATOR; // 1
    data[1] = 1; // version
    data[PROP_PROPOSAL_ID..PROP_PROPOSAL_ID + 32].copy_from_slice(proposal_id);
    // ... set all fields at correct offsets
    data
}
```

Verifying Results

After processing an instruction, read the resulting account data:

```
let result = mollusk.process_instruction(&ix, &accounts);
assert!(result.program_result.is_ok());

let prop_data = &result.resulting_accounts[0].1.data;
assert_eq!(read_u32(prop_data, PROP_YES_VOTES), 1);
assert_eq!(prop_data[PROP_STATUS], STATUS_OPEN);
```

Testing Double Vote Prevention

The VoteRecord PDA prevents double voting. If the PDA already exists, `CreateAccount` fails:

```
// Pre-populate VoteRecord (voter already voted)
let existing_vr = build_vote_record_data(&voter, &proposal_id, 1, vr_bump);

let result = mollusk.process_instruction(&ix, &[
    (proposal_pda, program_account(&program_id, proposal_data)),
    (vote_record_pda, program_account(&program_id, existing_vr)), // exists!
    // ...
]);
assert!(result.program_result.is_err());
```

Source

- Pinocchio tests: `chains/solana/examples/voting/pinocchio/tests/mollusk.rs`
- Native tests: `chains/solana/examples/voting/native/tests/mollusk.rs`

E2E Demo

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Prerequisites

- Rust toolchain with `cargo build-sbf`
- Bun (for TypeScript e2e)
- Program deployed to Solana devnet

Pre-Alpha Environment

Resource	Endpoint
dWallet gRPC	<code>https://pre-alpha-dev-1.ika.ika-network.net:443</code>
Solana RPC	<code>https://api.devnet.solana.com</code>

Quick Start

Deploy your voting program to devnet, then run:

```
# TypeScript
just e2e-voting <DWALLET_ID> <VOTING_ID>

# Rust
just e2e-voting-rust <DWALLET_ID> <VOTING_ID>
```

Available Demos

Command	Language	File
<code>just e2e-voting</code>	TypeScript (bun)	<code>examples/voting/e2e/main.ts</code>
<code>just e2e-voting-rust</code>	Rust	<code>examples/voting/e2e-rust/src/main.rs</code>

Both demos produce identical results — the TypeScript version uses shared helpers from `_shared/` .

What the Demo Does

- Step 1: gRPC DKG

Step 2: Transfer authority

Step 3: Create proposal

Step 4: Cast 3 YES votes

Step 5: Verify approval

Step 6: gRPC presign

Step 7: gRPC sign
- Creates dWallet, mock commits on-chain

→ Voting program CPI PDA owns the dWallet

→ quorum=3, message="Transfer 100 USDC to treasury"

→ Last vote triggers approve_message CPI

→ MessageApproval PDA exists on-chain

→ Allocate presign for signing

→ 64-byte Ed25519 signature returned

Expected Output

== dWallet Voting E2E Demo (TypeScript) ==

```
[Setup] Funding payer...
  ✓ Payer: ...
[Setup] Waiting for mock + creating dWallet via gRPC...
  ✓ DWalletCoordinator: ...
  ✓ dWallet on-chain: ...
  ✓ Authority transferred to CPI PDA: ...

[1/5] Creating voting proposal (quorum=3)...
  ✓ Proposal: ...
[2/5] Vote 1/3: Alice casts YES...
  ✓ Alice voted YES
[2/5] Vote 2/3: Bob casts YES...
  ✓ Bob voted YES
[2/5] Vote 3/3: Charlie casts YES...
  ✓ Charlie voted YES
  ✓ Proposal approved (3/3 yes)
[3/5] Verifying MessageApproval on-chain...
  ✓ MessageApproval: ...
[4/5] Allocating presign via gRPC...
  ✓ Presign allocated!
[5/5] Sending Sign request via gRPC...
  ✓ Signature received from gRPC!
  → Signature: <64-byte hex>
```

== E2E Test Passed! ==

How the Shared Helpers Work

The TypeScript e2e imports from `_shared/` :

```
import { setupDWallet, requestPresign, requestSign } from "../../_shared/ika-
setup.ts";
import { log, ok, val, sendTx, pda, pollUntil } from
 "../../_shared/helpers.ts";
```

`setupDWallet()` handles the entire dWallet lifecycle:

1. Waits for program initialization (polls for DWalletCoordinator PDA)
2. Sends gRPC DKG request (creates dWallet on-chain + transfers authority)
3. Polls for dWallet PDA to appear
4. Transfers authority to the example program's CPI PDA

This means the e2e test code only needs to focus on the voting-specific logic.

Multisig Example

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

An M-of-N multisig program controlling a dWallet. Fixed members are set at creation. Any member can propose transactions with message data stored on-chain for other members to inspect. Members approve or reject. When threshold approvals are reached, the program CPI-calls `approve_message` and optionally `transfer_future_sign`. When enough rejections accumulate (making approval impossible), the transaction is marked rejected.

Use case: Multi-party custody, organizational signing policies, timelocked operations.

Program Design

Accounts

Account	Seeds	Size	Description
Multisig	["multisig", create_key]	395 bytes	Members list, threshold, dWallet reference, tx counter
Transaction	["transaction", multisig, tx_index_le]	432 bytes	Message data, approval/rejection counts, status
ApprovalRecord	["approval", transaction,	68 bytes	Prevents double voting — one per member per

Account	Seeds	Size	Description
	member]		transaction

Instructions

Disc	Instruction	Description
0	CreateMultisig	Create multisig with members (up to 10), threshold, dWallet
1	CreateTransaction	Propose a transaction with message data stored on-chain
2	Approve	Approve; when threshold reached, CPI approve_message + optional transfer_future_sign
3	Reject	Reject; when enough rejections, mark transaction as rejected

Multisig Layout (395 bytes)

disc(1) + version(1) + create_key(32) + threshold(u16) + member_count(u16) + tx_index(u32) + dwallet(32) + bump(1) + members(32 * 10)

Transaction Layout (432 bytes)

disc(1) + version(1) + multisig(32) + tx_index(u32) + proposer(32) + message_hash(32) + user_pubkey(32) + signature_scheme(1) + approval_count(u16) + rejection_count(u16) + status(1) + message_approval_bump(1) + partial_user_sig(32) + bump(1) + message_data_len(u16) + message_data(256)

Key Offsets

Field	Offset	Size	Type
approval_count	135	2	u16 LE
rejection_count	137	2	u16 LE
status	139	1	0=Active, 1=Approved, 2=Rejected
message_data_len	174	2	u16 LE
message_data	176	256	raw bytes

Rejection Logic

A transaction is rejected when enough members reject that approval becomes impossible:

```
rejection_threshold = member_count - threshold + 1
```

For a 2-of-3 multisig: $3 - 2 + 1 = 2$ rejections needed to reject.

CPI Flow

When `approval_count >= threshold`, the program:

1. Calls `ctx.approve_message(...)` — creates `MessageApproval` PDA
2. If `partial_user_sig` is set (non-zero), calls `ctx.transfer_future_sign(...)` — transfers the partial signature completion authority to the proposer
3. Sets transaction status to `Approved`

E2E Flow

- | | |
|------------------------|---|
| 1. gRPC DKG | → dWallet created, authority = caller |
| 2. Transfer authority | → CPI PDA owns the dWallet |
| 3. Create multisig | → 2-of-3 with 3 member pubkeys |
| 4. Propose transaction | → message data stored on-chain |
| 5. Member1 approves | → <code>approval_count = 1</code> |
| 6. Member2 approves | → <code>approval_count = 2 = threshold</code> → CPI! |
| 7. Verify approval | → <code>MessageApproval</code> exists |
| 8. gRPC presign | → allocate presign |
| 9. gRPC sign | → 64-byte signature |
| 10. Rejection test | → propose 2nd tx, 2 rejections → <code>status=Rejected</code> |

React Frontend

A React frontend is included at `chains/solana/examples/multisig/react/` with:

- Create dWallet + Multisig (via gRPC-web client)
- Propose transactions
- Approve/reject as a member
- View transaction status and message data
- Airdrop button for local testing

```
cd chains/solana/examples/multisig/react && bun install && bun dev
```

Testing

```
# Mollusk (all 3 framework variants)
cargo test -p ika-example-multisig --test mollusk          # pinocchio (11
tests)
cargo test -p ika-example-multisig-native --test mollusk  # native (11 tests)

# TypeScript E2E
cd chains/solana/examples/multisig/e2e && bun main.ts <DWALLET_ID>
<MULTISIG_ID>
```

Source Files

- Pinocchio: chains/solana/examples/multisig/pinocchio/src/lib.rs
- Native: chains/solana/examples/multisig/native/src/lib.rs
- Anchor: chains/solana/examples/multisig/anchor/src/lib.rs
- TypeScript E2E: chains/solana/examples/multisig/e2e/main.ts
- React: chains/solana/examples/multisig/react/

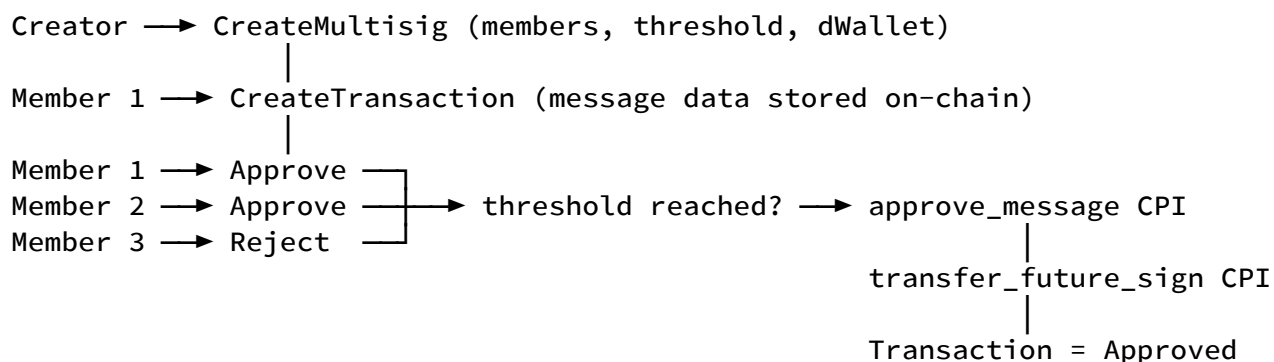
Building the Multisig Program

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What You’ll Learn

- How to design a multisig with fixed members and threshold approval
- How to store transaction data on-chain for other signers to inspect
- How to implement both approval and rejection flows
- How to use `transfer_future_sign` for partial signature management

Architecture



1. Account Layouts

Multisig PDA (["multisig", create_key]) — 395 bytes

Field	Offset	Size	Type
disc	0	1	always 1
version	1	1	always 1
create_key	2	32	unique key
threshold	34	2	u16 LE
member_count	36	2	u16 LE
tx_index	38	4	u32 LE (auto-increment)
dwallet	42	32	pubkey
bump	74	1	PDA bump
members	75	320	10 × 32-byte pubkeys

Transaction PDA (["transaction", multisig, tx_index_le]) — 432 bytes

Field	Offset	Size	Type
disc	0	1	always 2
multisig	2	32	pubkey
tx_index	34	4	u32 LE
proposer	38	32	pubkey
message_hash	70	32	keccak256
approval_count	135	2	u16 LE
rejection_count	137	2	u16 LE
status	139	1	0=Active, 1=Approved, 2=Rejected
message_data_len	174	2	u16 LE
message_data	176	256	raw bytes

ApprovalRecord PDA (["approval", transaction, member]) — 68 bytes

Prevents double voting. One per member per transaction.

2. Instructions

Disc	Name	Description
0	CreateMultisig	Set members (up to 10), threshold, dWallet reference
1	CreateTransaction	Propose with message data stored on-chain
2	Approve	Vote yes; triggers CPI at threshold
3	Reject	Vote no; marks rejected when impossible to approve

3. Rejection Threshold

A transaction is rejected when enough members reject that approval becomes impossible:

```
rejection_threshold = member_count - threshold + 1
```

Example: 2-of-3 multisig → $3 - 2 + 1 = 2$ rejections needed.

4. CPI Flow on Approval

When `approval_count >= threshold`:

```
// 1. Approve the message (creates MessageApproval PDA)
ctx.approve_message(
    message_approval, dwallet, payer, system_program,
    message_hash, user_pubkey, signature_scheme,
    message_approval_bump,
)?;

// 2. Optionally transfer future sign authority
if partial_user_sig != [0u8; 32] {
    ctx.transfer_future_sign(partial_user_sig_account, proposer_key)?;
}

// 3. Mark transaction as approved
tx_data[TX_STATUS] = STATUS_APPROVED;
```

Source Code

Framework	Path
Pinocchio	chains/solana/examples/multisig/pinocchio/src/lib.rs

Framework	Path
Native	chains/solana/examples/multisig/native/src/lib.rs
Anchor	chains/solana/examples/multisig/anchor/src/lib.rs

Testing the Multisig Program

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Test Matrix

Test	Instruction	
test_create_multisig_success	CreateMultisig	2 c
test_create_multisig_zero_threshold_fails	CreateMultisig	F t
test_create_multisig_threshold_exceeds_members_fails	CreateMultisig	F t r
test_create_transaction_success	CreateTransaction	M s i
test_create_transaction_non_member_fails	CreateTransaction	M c
test_approve_success	Approve	a i s
test_approve_double_vote_fails	Approve	A a
test_approve_non_member_fails	Approve	M c
test_reject_success	Reject	r i

Test	Instruction	
		s
test_reject_threshold_marks_rejected	Reject	2s
test_vote_on_closed_transaction_fails	Approve	(/ t

All 11 tests pass for both Pinocchio and Native variants.

Running Tests

```
# Pinocchio
cargo build-sbf --manifest-path
chains/solana/examples/multisig/pinocchio/Cargo.toml
cargo test -p ika-example-multisig --test mollusk

# Native
cargo build-sbf --manifest-path
chains/solana/examples/multisig/native/Cargo.toml
cargo test -p ika-example-multisig-native --test mollusk
```

Source

- Pinocchio: chains/solana/examples/multisig/pinocchio/tests/mollusk.rs
- Native: chains/solana/examples/multisig/native/tests/mollusk.rs

E2E Demo

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Pre-Alpha Environment

Resource	Endpoint
dWallet gRPC	<code>https://pre-alpha-dev-1.ika.ika-network.net:443</code>
Solana RPC	<code>https://api.devnet.solana.com</code>

Quick Start

Deploy your multisig program to devnet, then run:

```
# TypeScript
just e2e-multisig <DWALLET_ID> <MULTISIG_ID>

# Rust
just e2e-multisig-rust <DWALLET_ID> <MULTISIG_ID>
```

Available Demos

Command	Language	File
<code>just e2e-multisig</code>	TypeScript (bun)	<code>examples/multisig/e2e/main.ts</code>

Command	Language	File
<code>just e2e-multisig-rust</code>	Rust	<code>examples/multisig/e2e-rust/src/main.rs</code>

What the Demo Does

Step 1: gRPC DKG	→ dWallet created, authority = caller
Step 2: Transfer authority	→ CPI PDA owns the dWallet
Step 3: Create multisig	→ 2-of-3 with 3 member pubkeys
Step 4: Propose transaction	→ Message data stored on-chain
Step 5: Member1 approves	→ approval_count = 1
Step 6: Member2 approves	→ approval_count = 2 = threshold → CPI!
Step 7: Verify approval	→ MessageApproval exists on-chain
Step 8: gRPC presign	→ Allocate presign
Step 9: gRPC sign	→ 64-byte signature returned
Step 10: Rejection test	→ Propose 2nd tx, 2 rejections → Rejected

React Frontend

A React frontend is included for interactive testing:

```
cd chains/solana/examples/multisig/react
bun install && bun dev
```

The frontend includes:

- **Create dWallet + Multisig** in one click (via gRPC-web)
- **Propose transactions** with on-chain message data
- **Approve/Reject** as a connected wallet member
- **Live status** with auto-refresh

Instruction Reference

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Instructions in the Ika dWallet Solana program. The first byte of instruction data is the discriminator.

Instruction Groups

Group	Disc Range	Instructions
Message	8	approve_message
Ownership	24	transfer_ownership
DKG	31	commit_dwallet
Signing	42–43	transfer_future_sign, commit_signature

Message

approve_message (disc 8)

Create a `MessageApproval` PDA requesting a signature from the Ika network. Supports both direct signer and CPI callers.

Accounts (CPI path):

#	Account	W	S	Description
0	message_approval	yes	no	MessageApproval PDA (must be empty)
1	dwallet	no	no	dWallet account
2	caller_program	no	no	Calling program (executable)
3	cpi_authority	no	yes	CPI authority PDA (signed via invoke_signed)
4	payer	yes	yes	Rent payer
5	system_program	no	no	System program

Data (67 bytes):

Offset	Field	Size
0	discriminator	1
1	bump	1
2	message_hash	32
34	user_pubkey	32
66	signature_scheme	1

The dWallet program verifies:

- `caller_program` is executable
- `cpi_authority` matches `PDA(["__ika_cpi_authority"], caller_program)`
- `dwallet.authority == cpi_authority`

Ownership

transfer_ownership (disc 24)

Transfer dWallet authority to a new pubkey.

Accounts (signer path):

#	Account	W	S	Description
0	current_authority	no	yes	Current dWallet authority (signer)
1	dwallet	yes	no	dWallet account

Accounts (CPI path):

#	Account	W	S	Description
0	caller_program	no	no	Calling program (executable)
1	cpi_authority	no	yes	CPI authority PDA (signer)
2	dwallet	yes	no	dWallet account

Data (33 bytes):

Offset	Field	Size
0	discriminator	1
1	new_authority	32

DKG

commit_dwallet (disc 31)

NOA-only: create a dWallet account after DKG completes.

Accounts:

#	Account	W	S	Description
0	coordinator	no	no	DWalletCoordinator PDA
1	nek	no	no	NetworkEncryptionKey PDA
2	noa	no	yes	NOA signer
3	dwallet	yes	no	DWallet PDA (must be empty)
4	authority	no	no	Initial dWallet authority
5	payer	yes	yes	Rent payer
6	system_program	no	no	System program

Data:

Offset	Field	Size
0	discriminator	1
1	curve	1
2	is_imported	1
3	public_key_len	1
4	public_key	65
69	bump	1

Offset	Field	Size
70	public_output_len	2
72	public_output	256
328	noa_signature	64

Signing

transfer_future_sign (disc 42)

Transfer the completion authority of a `PartialUserSignature` .

Accounts (CPI path):

#	Account	W	S	Description
0	partial_user_sig	yes	no	PartialUserSignature account
1	caller_program	no	no	Calling program (executable)
2	cpi_authority	no	yes	CPI authority PDA (signer)

Data (33 bytes):

Offset	Field	Size
0	discriminator	1
1	new_completion_authority	32

commit_signature (disc 43)

NOA-only: write the signature into a `MessageApproval` account.

Accounts:

#	Account	W	S	Description
0	message_approval	yes	no	MessageApproval PDA
1	nek	no	no	NetworkEncryptionKey PDA
2	noa	no	yes	NOA signer

Data:

Offset	Field	Size
0	discriminator	1
1	signature_len	2
3	signature	128

Voting Example Instructions

These are defined by the example voting program, not the dWallet program:

`create_proposal` (disc 0)

Create a voting proposal.

Accounts:

#	Account	W	S	Description
0	proposal	yes	no	Proposal PDA (["proposal", proposal_id])
1	dwallet	no	no	dWallet account
2	creator	no	yes	Proposal creator
3	payer	yes	yes	Rent payer
4	system_program	no	no	System program

Data (103 bytes): proposal_id(32) | message_hash(32) | user_pubkey(32) | signature_scheme(1) | quorum(4) | message_approval_bump(1) | bump(1)

`cast_vote` (disc 1)

Cast a vote. Triggers `approve_message` CPI when quorum is reached.

Accounts (base, 5):

#	Account	W	S	Description
0	proposal	yes	no	Proposal PDA
1	vote_record	yes	no	VoteRecord PDA (["vote", proposal_id, voter])
2	voter	no	yes	Voter
3	payer	yes	yes	Rent payer

#	Account	W	S	Description
4	system_program	no	no	System program

Additional accounts when quorum reached (5):

#	Account	W	S	Description
5	message_approval	yes	no	MessageApproval PDA
6	dwallet	no	no	dWallet account
7	caller_program	no	no	Voting program
8	cpi_authority	no	no	CPI authority PDA
9	dwallet_program	no	no	dWallet program

Data (35 bytes): proposal_id(32) | vote(1) | vote_record_bump(1) |
cpi_authority_bump(1)

Account Reference

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

All account types in the Ika dWallet system. Each account starts with a 2-byte prefix: `discriminator(1) | version(1)`, followed by the account data.

dWallet Program Accounts

Account Discriminators

Discriminator	Account Type
1	DWalletCoordinator
2	DWallet
3	NetworkEncryptionKey
4	GasDeposit
9	PartialUserSignature
11	EncryptedUserSecretKeyShare
14	MessageApproval
15	DWalletAttestation

DWalletCoordinator (disc 1)

Program-wide state. PDA seeds: `["dwallet_coordinator"]`.

Offset	Field	Size	Description
0	discriminator	1	1
1	version	1	1
2	authority	32	Admin authority pubkey (NOA or multisig)
34	epoch	8	Current epoch number (LE u64)
42	total_dwallets_created	8	Total dWallets created (LE u64)
50	paused	1	Whether program is paused (0=no, 1=yes)
51	bump	1	PDA bump seed
52	_reserved	64	Reserved for future use

Total: 116 bytes (2 + 114)

NetworkEncryptionKey (disc 3)

The network encryption public key used for DKG. PDA seeds: ["network_encryption_key", noa_pubkey] .

Offset	Field	Size	Description
0	discriminator	1	3
1	version	1	1
2	(fields)	162	NEK data

Total: 164 bytes

DWallet (disc 2)

A distributed signing key. PDA seeds: ["dwallet", chunks_of(curve_u16_le || public_key)] – the curve u16 LE (2 bytes) is concatenated with the raw public key into a single buffer, then split into 32-byte pieces (Solana's MAX_SEED_LEN) and each chunk is passed as its own seed.

pubkey length	payload size	chunks
32 bytes (Ed25519 / Curve25519 / Ristretto)	34 bytes	[32, 2]
33 bytes (compressed Secp256k1 / Secp256r1)	35 bytes	[32, 3]
65 bytes (uncompressed SEC1)	67 bytes	[32, 32, 3]

Offset	Field	Size	Description
0	discriminator	1	2
1	version	1	1
2	authority	32	Who can approve messages (user or CPI PDA)
34	curve	2	Curve type (u16 LE): 0=Secp256k1, 1=Secp256r1, 2=Curve25519, 3=Ristretto
36	state	1	0=DKGInProgress, 1=Active, 2=Frozen
37	public_key_len	1	Actual key length (32 or 33)
38	public_key	65	dWallet public key (padded)
103	created_epoch	8	Epoch when created (LE u64)
111	noa_public_key	32	NOA Ed25519 public key used during DKG
143	is_imported	1	Whether the key was imported (0=standard, 1=imported)
144	bump	1	PDA bump seed
145	_reserved	8	Reserved for future use

Total: 153 bytes (2 + 151)

DWalletAttestation (disc 15)

Variable-size PDA storing BCS-serialized versioned attestation data + NOA Ed25519 signature. One per type per dWallet. Created by commit instructions (`CommitDWallet` , `CommitFutureSign` , `CommitEncryptedUserSecretKeyShare` , `CommitPublicUserSecretKeyShare`).

Offset	Field	Size	Description
0	discriminator	1	15
1	version	1	1
2	noa_signature	64	NOA Ed25519 signature over the attestation data
66	bump	1	PDA bump seed
67	attestation_data	variable	BCS-serialized versioned attestation struct

Header: 67 bytes. Total: 67 + len(attestation_data).

PDA seed patterns by type:

Type	Seeds
DKG	<code>["dwallet", chunks..., "attestation"]</code>
MakePublic	<code>["dwallet", chunks..., "public_user_share"]</code>
EncryptedShare (ReEncrypt)	<code>["dwallet", chunks..., "encrypted_user_share", &enc_key, "attestation"]</code>
FutureSign	<code>["dwallet", chunks..., "partial_user_sig", &scheme_u16_le, &msg_digest, [&meta_digest], "attestation"]</code>

GasDeposit (disc 4)

Per-user gas deposit. PDA seeds: `["gas_deposit", user_pubkey]` .

Offset	Field	Size	Description
0	discriminator	1	4
1	version	1	1
2	user_pubkey	32	Ed25519 public key for gRPC auth
34	ika_balance	8	Available IKA balance (LE u64)
42	sol_balance	8	Available SOL balance in lamports (LE u64)
50	total_ika_deposited	8	Lifetime IKA deposited
58	total_ika_consumed	8	Lifetime IKA consumed
66	total_sol_deposited	8	Lifetime SOL deposited
74	total_sol_consumed	8	Lifetime SOL consumed
82	pending_ika_withdrawal	8	Pending IKA withdrawal amount
90	pending_sol_withdrawal	8	Pending SOL withdrawal amount
98	withdrawal_epoch	8	Epoch when withdrawal becomes available (0=none)
106	last_settlement_epoch	8	Epoch of last gas settlement
114	created_at_epoch	8	Epoch when created
122	bump	1	PDA bump seed
123	_reserved	16	Reserved

Total: 139 bytes (2 + 137)

MessageApproval (disc 14)

A signing request. PDA seeds: ["dwallet", chunks..., "message_approval", &scheme_u16_le, &message_digest, [&message_metadata_digest]] .

The `message_metadata_digest` seed is only included when non-zero.

Offset	Field	Size	Description
0	discriminator	1	14
1	version	1	1
2	dwallet	32	dWallet account pubkey
34	message_digest	32	Keccak-256 digest of message to sign
66	message_metadata_digest	32	Keccak-256 digest of metadata (zero if none)
98	approver	32	dWallet authority who authorized signing
130	user_pubkey	32	User public key authorized for gRPC Sign
162	signature_scheme	2	DWalletSignatureScheme (u16 LE, values 0-6)
164	epoch	8	Epoch when approved (LE u64)
172	status	1	Pending(0) or Signed(1)
173	signature_len	2	Signature byte count (LE u16)
175	signature	128	Signature bytes (padded)
303	bump	1	PDA bump seed
304	_reserved	8	Reserved

Total: 312 bytes (2 + 310)

Status values:

- 0 = PENDING – awaiting signature from the network
- 1 = SIGNED – signature is available

PartialUserSignature (disc 9)

Partial user signature for the FutureSign flow. PDA seeds: ["dwallet", chunks..., "partial_user_sig", &scheme_u16_le, &message_digest, [&message_metadata_digest]] .

The `message_metadata_digest` seed is only included when non-zero.

Offset	Field	Size	Description
0	discriminator	1	9
1	version	1	1
2	dwallet	32	dWallet account pubkey
34	completion_authority	32	Authority that can complete the signature
66	message_digest	32	Keccak-256 digest of message
98	message_metadata_digest	32	Keccak-256 digest of metadata (zero if none)
130	signature_scheme	2	DWalletSignatureScheme (u16 LE)
132	partial_signature_len	2	Length of partial signature data (LE u16)
134	partial_signature	256	Partial signature from user
390	presign_id	32	Presign ID used
422	created_epoch	8	Epoch when created (LE u64)
430	status	1	Pending(0) or Signed(1)
431	signature_len	2	Final MPC signature length (LE u16)
433	signature	128	Final MPC signature (written by NOA)
561	bump	1	PDA bump seed
562	_reserved	8	Reserved

Total: 570 bytes (2 + 568)

EncryptedUserSecretKeyShare (disc 11)

Metadata for an encrypted user secret key share. PDA seeds: ["dwallet", chunks..., "encrypted_user_share", &encryption_key] .

Attestation data is stored in a separate `DWalletAttestation` PDA rooted under this share's seed hierarchy.

Offset	Field	Size	Description
0	discriminator	1	11
1	version	1	1
2	dwallet	32	dWallet account pubkey
34	encryption_key	32	Encryption key pubkey

Offset	Field	Size	Description
66	encryption_key_owner	32	Address of encryption key owner
98	source_share	32	Source share pubkey (zero if DKG-created)
130	is_re_encrypted	1	0=DKG, 1=re-encrypted
131	created_epoch	8	Epoch when created (LE u64)
139	bump	1	PDA bump seed
140	_reserved	8	Reserved

Total: 148 bytes (2 + 146)

PDA Seed Hierarchy

All dWallet-derived PDAs are rooted from the dWallet's `["dwallet", chunks(curve_u16_le || pk)]` prefix:

Account	Full PDA Seeds
DWallet	<code>["dwallet", chunks...]</code>
DKG attestation	<code>["dwallet", chunks..., "attestation"]</code>
MakePublic attestation	<code>["dwallet", chunks..., "public_user_share"]</code>
EncryptedShare	<code>["dwallet", chunks..., "encrypted_user_share", &enc_key]</code>
ReEncrypt attestation	<code>["dwallet", chunks..., "encrypted_user_share", &enc_key, "attestation"]</code>
MessageApproval	<code>["dwallet", chunks..., "message_approval", &scheme_u16_le, &message_digest, [&meta_digest]]</code>
PartialUserSignature	<code>["dwallet", chunks..., "partial_user_sig", &scheme_u16_le, &message_digest, [&meta_digest]]</code>
FutureSign attestation	<code>["dwallet", chunks..., "partial_user_sig", &scheme_u16_le, &msg_digest, [&meta_digest], "attestation"]</code>

The `[&meta_digest]` notation means the seed is only included when the message metadata digest is non-zero.

Ika System Accounts (SDK Types)

These accounts are part of the Ika System program, readable via `ika-solana-sdk-types`.

SystemState (disc 1)

PDA seeds: `["ika_system_state"]`. Total: **365 bytes**.

Offset	Field	Size	Description
0	discriminator	1	1
1	version	1	1
2	epoch	8	Current epoch (LE u64)
34	authority	32	System authority

Validator (disc 2)

PDA seeds: `["validator", identity_pubkey]`. Total: **973 bytes**.

Offset	Field	Size	Description
0	discriminator	1	2
1	version	1	1
2	identity	32	Validator identity pubkey
98	state	1	PreActive(0), Active(1), Withdrawing(2)
159	ika_balance	8	IKA token balance (LE u64)

StakeAccount (disc 3)

PDA seeds: `["stake_account", stake_id_le_bytes]`. Total: **115 bytes**.

Offset	Field	Size	Description
0	discriminator	1	3
1	version	1	1
2	owner	32	Stake owner pubkey
74	principal	8	Staked amount (LE u64)
98	state	1	Active(0), Withdrawing(1)

ValidatorList (disc 4)

PDA seeds: ["validator_list"] .

Offset	Field	Size	Description
0	discriminator	1	4
1	version	1	1
2	validator_count	4	Total validators (LE u32)
6	active_count	4	Active validators (LE u32)

Voting Example Accounts

Proposal (disc 1)

PDA seeds: ["proposal", proposal_id] . Total: **195 bytes**.

Offset	Field	Size	Description
0	discriminator	1	1
1	version	1	1
2	proposal_id	32	Unique identifier
34	dwallet	32	dWallet pubkey
66	message_hash	32	Message hash to sign
98	user_pubkey	32	User public key
130	signature_scheme	1	Signature scheme
131	creator	32	Creator pubkey
163	yes_votes	4	Yes count (LE u32)
167	no_votes	4	No count (LE u32)
171	quorum	4	Required yes votes (LE u32)
175	status	1	Open(0), Approved(1)
176	msg_approval_bump	1	MessageApproval PDA bump
177	bump	1	Proposal PDA bump
178	_reserved	16	Reserved

VoteRecord (disc 2)

PDA seeds: ["vote", proposal_id, voter] . Total: **69 bytes**.

Offset	Field	Size	Description
0	discriminator	1	2
1	version	1	1
2	voter	32	Voter pubkey
34	proposal_id	32	Proposal identifier
66	vote	1	Yes(1) or No(0)
67	bump	1	VoteRecord PDA bump

Account Type Summary

Account	Disc	Type	Size	PDA Seeds
DWalletCoordinator	1	PDA	116	["dwallet_coordinator"]
DWallet	2	PDA	153	["dwallet", chunks(curve_u16_le pk)]
NetworkEncryptionKey	3	PDA	164	["network_encryption_ noa"]
GasDeposit	4	PDA	139	["gas_deposit", user_pubkey]
PartialUserSignature	9	PDA	570	["dwallet", chunks... "partial_user_sig", .
EncryptedUserSecretKeyShare	11	PDA	148	["dwallet", chunks... "encrypted_user_share' &enc_key]
MessageApproval	14	PDA	312	["dwallet", chunks... "message_approval", .
DWalletAttestation	15	PDA	67+	["dwallet", chunks... <type-label>]
SystemState	1	PDA	365	["ika_system_state"]
Validator	2	PDA	973	["validator", identit
StakeAccount	3	PDA	115	["stake_account", stake_id]
ValidatorList	4	PDA	18+	["validator_list"]
Proposal	1	PDA	195	["proposal", id]

Account	Disc	Type	Size	PDA Seeds
VoteRecord	2	PDA	69	["vote", id, voter]

Instruction Discriminators

Instruction	Disc	Description
CreateDKGRequest	0	
CompleteDKGFirstRound	1	
SubmitUserDKGVerification	2	
CompleteDKG	3	
RejectDKG	4	
CreateImportedKeyDKGRequest	5	
CompleteImportedKeyVerification	6	
RejectImportedKeyVerification	7	
ApproveMessage	8	
CreatePresignRequest	11	
CompletePresign	12	
RejectPresign	13	
CreatePartialUserSignature	14	
VerifyPartialUserSignature	15	
RejectPartialUserSignature	16	
CreateEncryptionKey	17	
CreateEncryptedShare	18	
VerifyEncryptedShare	19	
RejectEncryptedShare	20	
AcceptEncryptedShare	21	
MakeUserSecretKeySharePublic	22	
VerifyMakePublic	23	
TransferOwnership	24	
CreateSigningDelegation	25	
CloseSigningDelegation	26	
RequestNetworkDKG	27	
CommitNetworkDKG	28	NOA commits network DKG result

Instruction	Disc	Description
RequestNetworkKeyReconfiguration	29	
CommitNetworkKeyReconfiguration	30	NOA commits key reconfiguration
CommitDWallet	31	NOA commits DKG result (creates DWallet + attestation PDA)
CommitFutureSign	33	NOA commits FutureSign (creates attestation PDA)
CommitEncryptedUserSecretKeyShare	34	NOA commits encrypted share (creates attestation PDA)
CommitPublicUserSecretKeyShare	35	NOA commits public share (creates attestation PDA)
CreateDeposit	36	
TopUp	37	
SettleGas	38	
UpdateFees	39	
PauseCurve	40	
UnpauseCurve	41	
TransferFutureSign	42	
CommitSignature	43	NOA writes signature (dispatches to MessageApproval or PartialUserSignature by discriminator)
RequestWithdraw	44	
Withdraw	45	
Initialize	46	
EmitEvent	228	Self-CPI event handler

Event Reference

Pre-Alpha Disclaimer: This is a pre-alpha release for development and testing only. Signing uses a single mock signer, not real distributed MPC. All 11 protocol operations are implemented (DKG, Sign, Presign, FutureSign, ReEncryptShare, etc.) across all 4 curves and 7 signature schemes, but without real MPC security guarantees. The dWallet keys, trust model, and signing protocol are not final; do not rely on any key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Ika Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

The dWallet program emits events via Anchor-compatible self-CPI. Events are emitted as inner instructions and can be parsed from transaction metadata.

Anchor-Compatible Event Format

Events use the same wire format as Anchor events:

```
EVENT_IX_TAG_LE(8) | event_discriminator(1) | event_data(N)
```

The `EVENT_IX_TAG_LE` is 8 bytes (`0xe4a545ea51cb9a1d` in little-endian). The event discriminator follows, then the event-specific data.

Key Events

MessageApprovalCreated

Emitted when `approve_message` creates a new `MessageApproval` PDA.

Field	Size	Description
<code>dwallet</code>	32	dWallet pubkey

Field	Size	Description
message_hash	32	Hash of the message to sign
caller_program	32	Program that approved

The Ika network listens for this event to initiate the signing protocol.

SignatureCommitted

Emitted when the NOA calls `commit_signature` to write a signature.

Field	Size	Description
message_approval	32	MessageApproval account pubkey
signature_len	2	Length of the signature

Off-chain clients can listen for this to know when a signature is ready.

DWalletCreated

Emitted when `commit_dwallet` creates a new dWallet.

Field	Size	Description
dwallet	32	New dWallet pubkey
authority	32	Initial authority
curve	1	Curve identifier

AuthorityTransferred

Emitted when `transfer_ownership` changes a dWallet's authority.

Field	Size	Description
dwallet	32	dWallet pubkey
old_authority	32	Previous authority
new_authority	32	New authority

Parsing Events

Events appear as inner instructions in the transaction metadata. To parse them:

1. Find inner instructions targeting the dWallet program
2. Match the first 8 bytes against `EVENT_IX_TAG_LE`
3. Read the 1-byte event discriminator
4. Deserialize the remaining bytes according to the event schema

Example: Detecting Signatures

```
use solana_transaction_status::UiTransactionEncoding;

let tx = client.get_transaction_with_config(
    &tx_signature,
    RpcTransactionConfig {
        encoding: Some(UiTransactionEncoding::Base64),
        commitment: Some(CommitmentConfig::confirmed()),
        max_supported_transaction_version: Some(0),
    },
)?;

// Parse inner instructions for SignatureCommitted events
if let Some(meta) = tx.transaction.meta {
    for inner_ix in meta.inner_instructions.unwrap_or_default() {
        for ix in inner_ix.instructions {
            // Check EVENT_IX_TAG_LE prefix and parse event data
        }
    }
}
```

Example: Polling for MessageApproval Status

Rather than parsing events, you can poll the `MessageApproval` account directly:

```
loop {
    let data = client.get_account(&message_approval_pda)?.data;
    if data[139] == 1 { // status == Signed
        let sig_len = u16::from_le_bytes(data[140..142].try_into().unwrap()) as
usize;
        let signature = data[142..142 + sig_len].to_vec();
        break;
    }
    std::thread::sleep(Duration::from_millis(500));
}
```

Event vs Polling

Approach	Pros	Cons
Event parsing	Immediate notification, no polling	Requires transaction metadata, more complex
Account polling	Simple, works everywhere	Latency, wasted RPC calls

For production use, event-based detection is recommended. For testing and simple scripts, polling is sufficient.